

Week10 – Lab09: Serial Communication

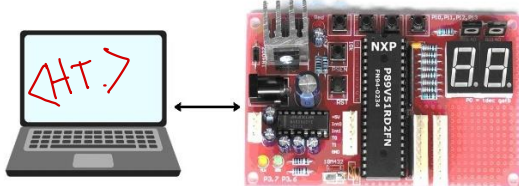
PC ____ รหัส _____ ชื่อ-สกุล _____

เปลี่ยนชื่อไฟล์เป็น “B6501234-นายไมโคร รายงานแลป-Lab09.pdf”

ส่งงาน Report ที่ >> <https://forms.gle/vYev4w6b7ZZuXgKdA>

B3701234 Wiclai

- MCS51 - Tx: จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ขึ้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11
- MCS51 - Rx:
 - จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง - (ขีดกลาง)
 - ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ, ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ
 - ถ้ารับค่าคีย์เป็น 0, +, - หากเป็น { + , 0 , - } ให้ 7Segment { นับขึ้น คงค่าการนับ นับลง }
- MCS51 - Tx, Rx:
 - ทดสอบส่งค่าผ่าน Serial-1 ด้วยค่าการกดสวิตช์ SW0 SW1 SW2 SW3 หากกด SW0 ให้ส่ง ASCII ของเลข 0, กด SW1 = “1”, กด SW2 = “2” และ กด SW3 = “3”
 - ทดสอบรับค่าจาก Serial-1 หากรับ ASCII เลข 0-9 ให้แสดงเลข 0-9 ที่ 7Segment
 - ต่อสาย TX จากบอร์ด1 ไปยัง Rx ของบอร์ด2 และ GND-GND และทดสอบการทำงาน

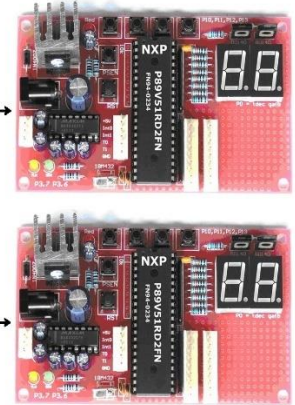
Mission.01
Mission.02PC ← MCS51
PC → MCS51

(M1) → Tx (พด)

(M2) → Rx (พง)

Mission.03

MCS51 → MCS51



1.MCS51 Tx	2.MCS51 Rx	3.MCS51 Tx-Rx	4.ESP32 OTA
5.ESP32 Rx	6.ESP32 Tx + Rx	7. ESP32 -- Modbus (1)	8. ESP32 -- Modbus (2)

4. ESP32 - OTA: ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี

- Test41 - Arduino Basic OTA
- Test42 - Arduino Web Updater OTA
- Test43 - Arduino IAP (In Application Programming)
- Test43 - Blynk OTA

Blink01	Blink02
<pre> 1 2 void setup() { 3 pinMode(LED_BUILTIN, OUTPUT); 4 } 5 6 void loop() { 7 digitalWrite(LED_BUILTIN, HIGH); delay(500); 8 digitalWrite(LED_BUILTIN, LOW); delay(500); 9 } 10 </pre>	<pre> 1 2 void setup() { 3 pinMode(LED_BUILTIN, OUTPUT); 4 } 5 6 void loop() { 7 digitalWrite(LED_BUILTIN, HIGH); delay(250); 8 digitalWrite(LED_BUILTIN, LOW); delay(250); 9 digitalWrite(LED_BUILTIN, HIGH); delay(250); 10 digitalWrite(LED_BUILTIN, LOW); delay(1000); 11 } 12 </pre>

5. ESP32 - Rx:

- จากโปรแกรมตัวอย่าง
- ให้ปรับเป็นโปรแกรมสร้างสัญญาณ โดยรับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ
P=xxx<Cr>, F=yy<Cr>, G, ?
- Command "G" = go เพื่อส่งความถี่และจำนวนพัลส์
- "?" = Help เพื่ออธิบายการใช้คำสั่ง แจ้งชื่อผู้พัฒนาโปรแกรม แจ้งเวอร์ชันโปรแกรม

Read String and Split → P=xxx, F=yyy, G

```

1 String Cmd, strData;
2 int strInt, strlength, Pulse = 0, Freq = 0;
3 char buffer[40];
4
5 void setup() {
6   Serial.begin(115200);
7 }
8
9 void loop() {
10  if (Serial.available() > 0) {
11    strData = Serial.readString();
12    strlength = strData.length();
13    Cmd = strData.substring(0, 1);
14    strInt = strData.substring(2, strlength).toInt();
15
16    if (Cmd == "F") {
17      if (strInt != 0) Freq = strInt;
18      sprintf(buffer, "read F = %S --> %d", strData, strInt);
19      Serial.println(buffer);
20    }
21
22    if (Cmd == "P") {
23      if (strInt != 0) Pulse = strInt;
24      sprintf(buffer, "read P = %S --> %d", strData, strInt);
25      Serial.println(buffer);
26    }
27
28    sprintf(buffer, "Pulse=%d, Freq=%d, Cmd=%s", Pulse, Freq, Cmd);
29    Serial.println(buffer);
30  }
31 }
32

```

COM3

F=1234

read F = F=1234 --> 1234
Pulse=0, Freq=1234, Cmd=F
read P = P=5678 --> 5678
Pulse=5678, Freq=1234, Cmd=P
Pulse=5678, Freq=1234, Cmd=G

☒ Autoscroll ☐ Show timestamp

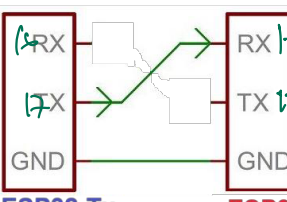
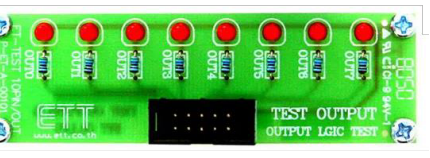
No line ending

115200 baud

Clear output

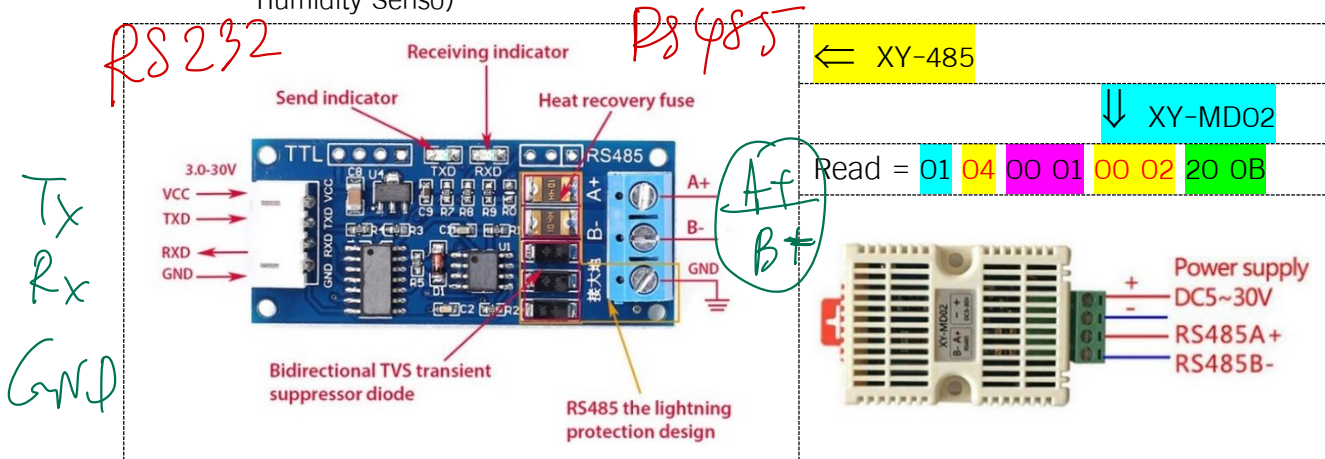
6. ESP32 - Tx, Rx: ให้นักศึกษากับคู่ระหว่างเครื่องเลขคี่ กับ เลขคี่+1 ทำการเขียนโปรแกรมร่วมกัน ดังนี้

- Read <https://www.artronshop.co.th/article/122/how-to-use-serial1-serial2-on-esp32>
- ทดสอบโปรแกรมตัวอย่าง ที่ให้มา
- PC-คู่ ให้เป็นคนส่ง: ตัวอย่างจตาม “Digital Input – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล
- PC-คี่ ให้เป็นคนรับ: ตัวอย่างจตาม “Digital Output – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล

		
ESP32-Tx UART2 Tx = 17	ESP32-Rx UART2 Rx = 16	
Tx.ino <pre> 1 #define LED_Test 2 2 #define SW_Test 0 3 #define Pin_RX2 16 ✓ 4 #define Pin_TX2 17 ✓ 5 6 int swRead; 7 8 void setup() { 9 Serial.begin(115200); 10 Serial2.begin(9600); 11 Serial2.setPins(Pin_RX2, Pin_TX2); 12 pinMode(LED_Test, OUTPUT); 13 pinMode(SW_Test, INPUT_PULLUP); 14 } 15 16 void loop() { 17 swRead = digitalRead(SW_Test); 18 Serial.println(swRead); 19 Serial2.println(swRead); 20 delay(100); 21 } 22 </pre>	Rx.ino <pre> 1 #define LED_Test 2 2 #define SW_Test 0 3 #define Pin_RX2 16 ✓ 4 #define Pin_TX2 17 ✓ 5 6 int keyInput; 7 8 void setup() { 9 Serial.begin(115200); 10 Serial2.begin(9600); 11 Serial2.setPins(Pin_RX2, Pin_TX2); 12 pinMode(LED_Test, OUTPUT); 13 pinMode(SW_Test, INPUT_PULLUP); 14 } 15 16 void loop() { 17 if (Serial2.available() > 0) { 18 keyInput = Serial2.parseInt(); 19 Serial.println(keyInput); 20 digitalWrite(LED_Test, keyInput); 21 } 22 } 23 </pre>	

7. ESP32 – Modbus Master:

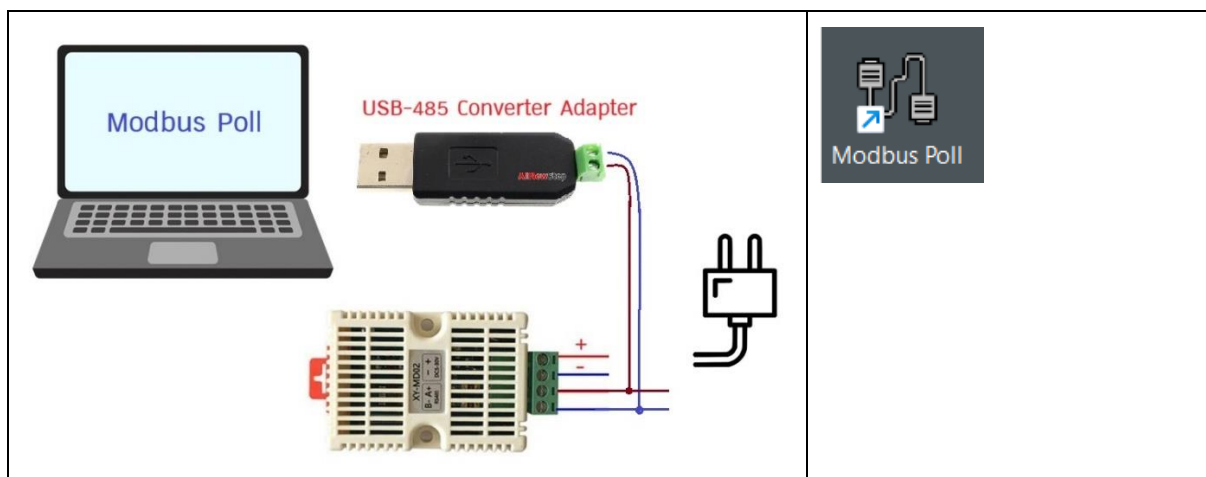
- Read <https://iot-kmutnb.github.io/blogs/sensors/xy-md02/>
- Read <https://modbus-tool.arttronshop.co.th/>
- Read <https://www.cybertice.com/p/4530>
- ทำความเข้าใจ และทดสอบโปรแกรมเพื่ออ่านค่า XY-MD02 (Low Cost RS485 Temperature & Humidity Senso)



7.1 Modbus

Modbus protocol:	XY-MD02 Memory Map																								
0x03: Read keep register 0x04: Read input register 0x06: Write a single keep register 0x10: Write more keep registers	<table><tr><th>Register Type</th><th>Register Address</th><th>Register contents</th><th>Number of bytes</th></tr><tr><td rowspan="2">Input Register</td><td>0x0001</td><td>Temperature</td><td>2</td></tr><tr><td>0x0002</td><td>Humidity</td><td>2</td></tr><tr><td rowspan="4">Keep Register</td><td>0x0101</td><td>Device Address</td><td>2</td></tr><tr><td>0x0102</td><td>Baud Rate 0:9600 1:14400 2:19200</td><td>2</td></tr><tr><td>0x0103</td><td>Temperature correction(/10) -10.0~10.0</td><td>2</td></tr><tr><td>0x0104</td><td>Humidity correction(/10) -10.0~10.0</td><td>2</td></tr></table>	Register Type	Register Address	Register contents	Number of bytes	Input Register	0x0001	Temperature	2	0x0002	Humidity	2	Keep Register	0x0101	Device Address	2	0x0102	Baud Rate 0:9600 1:14400 2:19200	2	0x0103	Temperature correction(/10) -10.0~10.0	2	0x0104	Humidity correction(/10) -10.0~10.0	2
Register Type	Register Address	Register contents	Number of bytes																						
Input Register	0x0001	Temperature	2																						
	0x0002	Humidity	2																						
Keep Register	0x0101	Device Address	2																						
	0x0102	Baud Rate 0:9600 1:14400 2:19200	2																						
	0x0103	Temperature correction(/10) -10.0~10.0	2																						
	0x0104	Humidity correction(/10) -10.0~10.0	2																						

7.2 Test with Modbus Poll



Read/Write Definition

Slave ID: 1

Function: 04 Read Input Registers (3x)

Address: 1

Quantity: 2

Scan Rate: 1000 ms

☒ Read/Write Enabled

View

Rows

☒ 10
 ☐ 20
 ☐ 50
 ☐ 100

☐ Hide Alias Columns
 ☐ Address in Cell
 ☐ PLC Addresses (Base 1)

Display: Unsigned

OK

Cancel

Apply

Slave ID = 1

Function = 04

Address = 1

Quantity = 2

Display = Unsigned

Modbus Poll - Mbpoll1

File

Edit

Connection

Setup

Functions

Display

View

Window

Help

05

06

15

16

22

23

101

?

?

Mbpoll1

Tx = 20: Err = 0: ID = 1: F = 04: SR = 1000ms

	Alias	00000
0		
1		262
2		583
3		
4		
5		
6		
7		
8		
9		

Communication Traffic

Exit

Stop

Save

```

000012-Tx:01 04 00 01 00 02 20 08
000013-Rx:01 04 04 01 04 02 41 7A E9
000014-Tx:01 04 00 01 00 02 20 08
000015-Rx:01 04 04 01 04 02 42 3A E8
000016-Tx:01 04 00 01 00 02 20 08
000017-Rx:01 04 04 01 04 02 42 3A E8
000018-Tx:01 04 00 01 00 02 20 08
000019-Rx:01 04 04 01 04 02 42 3A E8
000020-Tx:01 04 00 01 00 02 20 08
000021-Rx:01 04 04 01 05 02 42 6B 28
000022-Tx:01 04 00 01 00 02 20 08
000023-Rx:01 04 04 01 05 02 42 6B 28
000024-Tx:01 04 00 01 00 02 20 08
000025-Rx:01 04 04 01 06 02 42 9B 28
000026-Tx:01 04 00 01 00 02 20 08
000027-Rx:01 04 04 01 05 02 43 AA E8
000028-Tx:01 04 00 01 00 02 20 08
000029-Rx:01 04 04 01 05 02 43 AA E8
000030-Tx:01 04 00 01 00 02 20 08
000031-Rx:01 04 04 01 05 02 43 AA E8
000032-Tx:01 04 00 01 00 02 20 08
000033-Rx:01 04 04 01 05 02 44 BA 2A
000034-Tx:01 04 00 01 00 02 20 08
000035-Rx:01 04 04 01 05 02 45 2A EA
000036-Tx:01 04 00 01 00 02 20 08
000037-Rx:01 04 04 01 05 02 46 3A E8
000038-Tx:01 04 00 01 00 02 20 08
000039-Rx:01 04 04 01 06 02 47 6B 28
          
```

7.3 Circuit and Add Arduino Library

LIBRARY MANAGER

ModbusMaster

Type: All

Topic: All

ModbusMaster by Doc Walker

Enlighten your Arduino to be a Modbus master. Enables communication with Modbus slaves over RS232/485 (via RTU protocol). Requires an RS232/485 transceiver.

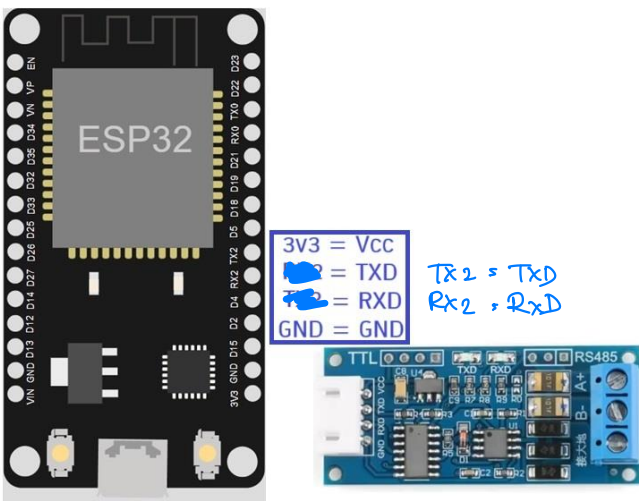
More info

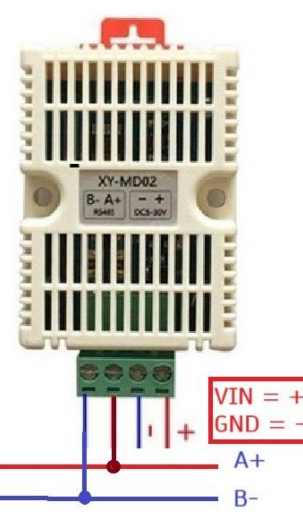
2.0.1

INSTALL

7.4 Test with ESP32

Master





Modbus.ino

```

1  #include "ModbusMaster.h" //https://github.com/4-20ma/ModbusMaster
2  #define Slave_ID 1
3  #define RX_PIN 16
4  #define TX_PIN 17
5
6  ModbusMaster modbus;
7
8  void setup() {
9      Serial.begin(115200, SERIAL_8N1);
10     Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
11     modbus.begin(Slave_ID, Serial2);
12 }
13
14 long lastMillis = 0;
15 double temp_dbl, humid_dbl;
16 String resString;
17
18 void loop() {
19     long currentMillis = millis();
20     if (currentMillis - lastMillis > 1000) {
21         resString = "\r\n";
22         uint8_t result = modbus.readInputRegisters(1, 2);
23         if (result == modbus.ku8MBSuccess) {
24             temp_dbl = modbus.getResponseBuffer(0) / 10.0;
25             humid_dbl = modbus.getResponseBuffer(1) / 10.0;
26             resString += "Temperature: " + String(temp_dbl) + " C\r\n";
27             resString += "Humidity: " + String(humid_dbl) + " %";
28         } else {
29             resString += "Modbus Read Error";
30             Serial.println(resString);
31             lastMillis = currentMillis;
32         }
33     }
34 }

```

```

#include "ModbusMaster.h"
//https://github.com/4-20ma/ModbusMaster
#define Slave_ID 1
#define RX_PIN 16
#define TX_PIN 17

ModbusMaster modbus;

void setup() {
    Serial.begin(115200, SERIAL_8N1);
    Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
    modbus.begin(Slave_ID, Serial2);
}

long lastMillis = 0;
double temp_dbl, humid_dbl;
String resString;

void loop() {
    long currentMillis = millis();
    if (currentMillis - lastMillis > 1000) {
        resString = "\r\n";
        uint8_t result = modbus.readInputRegisters(1, 2);
        if (result == modbus.ku8MBSuccess) {
            temp_dbl = modbus.getResponseBuffer(0) / 10.0;
            humid_dbl = modbus.getResponseBuffer(1) / 10.0;
            resString += "Temperature: " + String(temp_dbl) + " C\r\n";
            resString += "Humidity: " + String(humid_dbl) + " %";
        } else {
            resString += "Modbus Read Error";
            Serial.println(resString);
            lastMillis = currentMillis;
        }
    }
}

```

Output Serial Monitor X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM6')

```

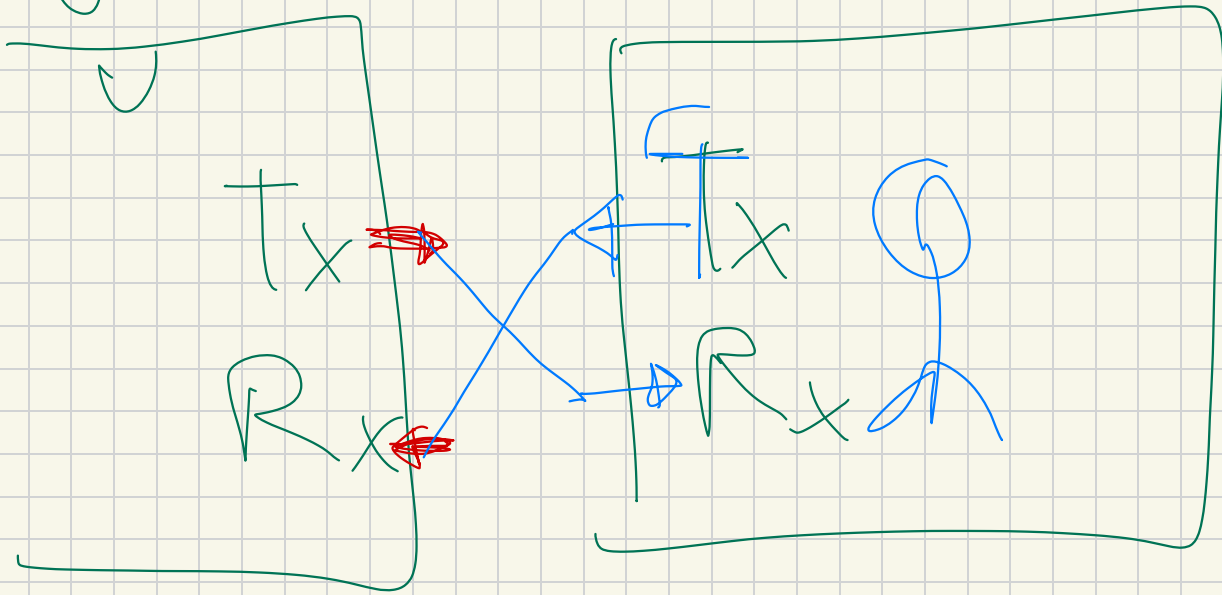
-----
Humidity: 55.00 %

Temperature: 27.10 C
Humidity: 55.00 %

Temperature: 27.10 C
Humidity: 55.10 %

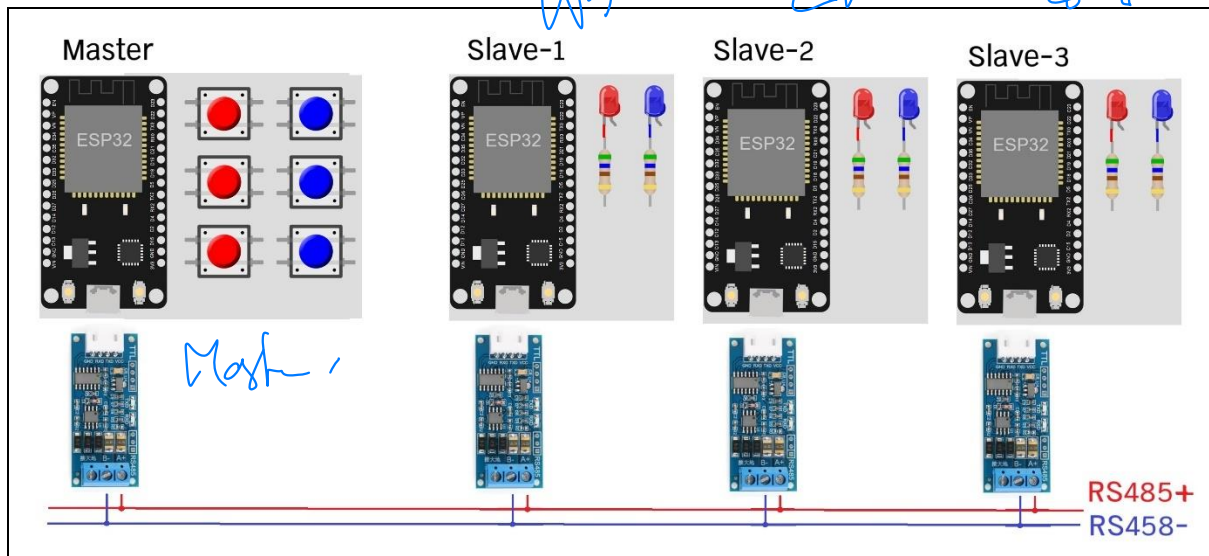
```

CP



8. ESP32 - Modbus Master & Slave:

- Read <https://embeddedthere.com/multiple-esp32-communication-via-rs485/>
- ปรับโปรแกรมเพื่อทดสอบการทำงาน



งาน

Modbus

2/7: Report Frame

1. MCS51 - Tx: จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปล่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ขึ้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11

Picture(รูปถ่ายการทำงาน)
MCS51 Capture Code
MCS51 Text Code

2. MCS51 - Rx:

- จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง - (ขีดกลาง)
- ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ, ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ
- ถ้ารับค่าคีย์เป็น 0, +, - หากเป็น {+, 0, -} ให้ 7Segment { นับขึ้น คงค่าการนับ นับลง }

Picture(รูปถ่ายการทำงาน)
MCS51 Capture Code
MCS51 Text Code

3. MCS51 - Tx, Rx:

- ทดสอบส่งค่าผ่าน Serial-1 ด้วยค่าการกดสวิตช์ SW0 SW1 SW2 SW3 หากกด SW0 ให้ส่ง ASCII ของเลข 0, กด SW1 = “1”, กด SW2 = “2” และ กด SW3 = “3”
- ทดสอบรับค่าจาก Serial-1 หากรับ ASCII เลข 0-9 ให้แสดงเลข 0-9 ที่ 7Segment
- ต่อสาย Tx จากบอร์ด1 ไปยัง Rx ของบอร์ด2 และ GND-GND และทดสอบการทำงาน

Picture(รูปถ่ายการทำงาน)
MCS51 Capture Code
MCS51 Text Code

4 ESP32 - OTA: ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี

Picture(รูปถ่ายการทำงาน)
Capture หน้าจอขณะ Upload

5 ESP32 - Rx: เขียนโปรแกรมสร้างสัญญาณ โดยรับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ
P=xxx<Cr>, D=yy<Cr>, G, ?

Picture(รูปถ่ายการทำงาน)
ESP32 Capture Code
ESP32 Text Code

6 ESP32 - Tx, Rx: ให้นักศึกษากับคู่ระหว่างเครื่องเลขคี่ กับ เลขคี่+1 ทำการเขียนโปรแกรมร่วมกัน ดังนี้

- PC-คู่ ให้เป็นคนส่ง: ต่อดิจิตอลตาม “Digital Input – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล
- PC-คี่ ให้เป็นคนรับ: ต่อดิจิตอลตาม “Digital Output – 8 LED”, ใช้ Serial2 ในการรับส่งข้อมูล

Picture(รูปถ่ายการทำงาน)
ESP32-Tx Capture Code
ESP32-Tx Text Code
ESP32-Rx Capture Code
ESP32-Rx Text Code

7 ESP32 - Modbus Master: Read XY-MD02 (Low Cost RS485 Temperature & Humidity Senso)

Picture(รูปถ่ายการทำงาน)
ESP32-Master Capture Code
ESP32-Master Text Code

8 ESP32 - Modbus Master&Slave: M

Picture(รูปถ่ายการทำงาน)
ESP32-Master Capture Code
ESP32-Master Text Code
ESP32-Slave Capture Code
ESP32-Slave Text Code

2/7: Serial Communication

00-Serial Communication

01-OSI Model and Industrial Communication Protocol

02-Asynchronous RS232 RS422 RS485

< Insert >

3/7: MCS51 Serial Communication

1. รีจิสเตอร์ที่เกี่ยวข้อง

TMOD	G(T1)	C/T	M1	M0	G(T0)	C/T	M1	M0
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
SCON	SM0	SM1	SM2	REN	TB8	RB8	TI	RI
PCON	SMOD	-	-	-	GF1	GF0	PD	IDL
T2CON	TF2	EXF2	RCLK	TCLK	EXEN2	TR2	C/T2	CP/RL2

<https://studylib.net/doc/13957472/8051-family-special-function-registers--pcon-%E2%80%93-power-cont...>

PCON – Power Control Register

D7	D6	D5	D4	D3	D2	D1	D0
SMOD	x	x	x	GF1	GF0	PD	IDL

Address: 87H (not bit addressable)

SMOD – Serial mode bit used to determine the baud rate with Timer 1.

SMOD = 0

$$\text{Baud rate} = \frac{\text{Oscillator frequency in Hz}}{N[256 - (TH1)]}$$

If SMOD = 0 then N = 384. If SMOD = 1 then N = 192. TH1 is the high byte of timer 1 when it is in 8-bit autoreload mode.

GF1 and GF0 are General purpose flags not implemented on the standard device

PD is the power down bit. Not implemented on the standard device

IDL activate the idle mode to save power. Not implemented on the standard device

TCON – Timer Control Register

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

Address: 88H (bit addressable)

TF1 – Timer 1 overflow flag

TR1 – Timer 1 run control bit

TF0 – Timer 0 overflow flag

TR0 – Timer 0 run control bit

IE1 – External interrupt 1 edge flag. Set to 1 when edge detected.

IT1 – Edge control bit for external interrupt 1. 1 = edge, 0 = level

IE0 – External interrupt 0 edge flag. Set to 1 when edge detected

IT0 – Edge control bit for external interrupt 0. 1 = edge, 0 = level

SCON – Serial Control Register

D7	D6	D5	D4	D3	D2	D1	D0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Address: 98H (bit-addressable)

SM0 SM1 Operation Baud rate

0	0	Shift register	Osc/12
0	1	8-bit UART	Set by timer
1	0	9-bit UART	Osc/12 or Osc/64
1	1	9-bit UART	Set by timer

SM2 – Enables multiprocessor communication in modes 2 and 3.

REN – Receiver enable

TB8 – Transmit bit 8. This is the 9th bit transmitted in modes 2 and 3.

RB8 – Receive bit 8. This is the 9th bit received in modes 2 and 3.

TI – Transmit interrupt flag. Set at end of character transmission. Cleared in software.

RI – Receive interrupt flag. Set at end of character reception. Cleared in software.

TMOD – Timer Mode Control Register

D7	D6	D5	D4	D3	D2	D1	D0
Gate	$\overline{C/T}$	M1	M0	Gate	$\overline{C/T}$	M1	M0
Timer 1				Timer 0			

Gate – if 1 timer x is enabled when intx is high and TRx is high. if 0 timer x is enabled when TRx is high.

$\overline{C/T}$ - if 1 timer x is clocked from Tx pin. if 0 timer x is clocked from oscillator/12

M1 M0 Mode

0	0	13-bit mode for compatibility to 8048 family
0	1	16-bit Timer/Counter. User must reload in software
1	0	8-bit autoreload. TLx is automatically reloaded from THx
1	1	TL0 is 8-bit counter controlled by Timer0 control bits. TH0 is 8-bit counter controlled by Timer1 control bits. Timer 1 is stopped

T2CON – Timer 2 Control Register

D7	D6	D5	D4	D3	D2	D1	D0
TF2	EXF2	RCLK	TCLK	EXEN2	TR2	$\overline{C/T2}$	$\overline{CP/RL2}$

Address: 0C8H (bit addressable)

TF2 – Timer 2 overflow flag

EXF2 – Timer 2 external flag.

RCLK – Receive clock. When set causes the serial port to use timer 2 for reception.

TCLK – Transmit clock. When set causes the serial port to use timer 2 for transmission.

$\overline{C/T2}$ – Counter/Timer select. if 0 use internal timer. if 1 use external pin

$\overline{CP/RL2}$ – Capture/reload flag.

IE – Interrupt Enable Register

D7	D6	D5	D4	D3	D2	D1	D0
EA	x	ET2	ES	ET1	EX1	ET0	EX0

Address: 0A8H (bit addressable)

EA – Global interrupt enable

x – not defined

ET2 – Timer 2 interrupt enable

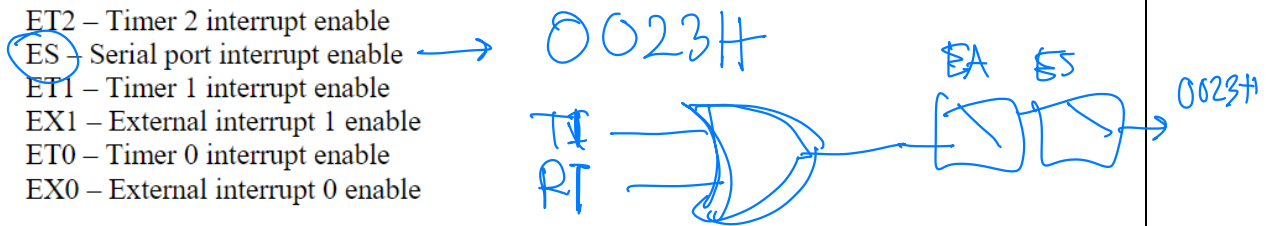
ES – Serial port interrupt enable

ET1 – Timer 1 interrupt enable

EX1 – External interrupt 1 enable

ET0 – Timer 0 interrupt enable

EX0 – External interrupt 0 enable

**IP – Interrupt Priority Register**

D7	D6	D5	D4	D3	D2	D1	D0
x	x	PT2	PS	PT1	PX1	PT0	PX0

Address: 0B8H (bit addressable)

x – not defined

PT2 – Priority for timer 2 interrupt

PS – Priority for serial port interrupt

PT1 – Priority for timer 1 interrupt

PX1 – Priority for external interrupt 1

PT0 – Priority for timer 0 interrupt

PX0 – Priority for external interrupt 0

2. เริ่มต้นใช้งาน Serial Communication

- a. Data Frame → SCON
- b. Baud Rate → TMOD, TCON, PMOD, T2CON
- c. Interrupts → IE, IP

Using Timer1, Mode2

<http://www.intel.com/design/mcs51/aplnots/270490.htm>

Serial Mode1,3 และใช้ Timer 1 Mode2 →

$$\text{BaudRate} = \frac{2^{\text{SMOD}} \times \text{Oscillator Frequency}}{32 \times 12 \times [256 - (\text{TH1})]}$$

Handwritten notes: 9600, 12000000, 11.0592 x 10^6, SMOD1

9600 @11.0592MHz
9600 @18.432MHz
19200 @18.432MHz

→ SMOD1=0, TH1=253(FDH)
→ SMOD1=0, TH1=251(FBH) (X1 Mode)
→ SMOD1=0, TH1=251(FBH) (X2 Mode)

```
MOV SCON,#050H ; Serial Port Mode 3
ANL PCON,#7FH ; SMOD 1=2
MOV TMOD,#20H ; Timer1 Mode2
MOV TH1,#0FDH ; Reload value for desired baud
SETB TR1 ; Turn on Timer1
```

Handwritten notes: TH1 = 253, TH1 = 252.7, 253, 252

Serial begin(9600);

Using Timer2

$$\text{Baud rate} = \frac{\text{Oscillator Frequency}}{32 \times [65536 - (\text{RCAP2H}, \text{RCAP2L})]}$$

9600 @11.0592MHz RCAP2H,RCAP2L = 65500 = FFDCH
9600 @18.432MHz RCAP2H,RCAP2L = 65476 = FFC4H (X1 Mode)
19200 @18.432MHz RCAP2H,RCAP2L = 65476 = FFC4H (X2 Mode)

```
MOV SCON,#01010000B ; Mode1 Serial Port
MOV RCAP2H,#0FFH ; Reload values for desired
MOV RCAP2L,#0DCH ; baud rate 9600@11.0592
MOV T2CON,#00110100B ; Timer 2 as baud rate generator
; and turn on Timer 2
```

T2CON EQU 0C8H
RCAP2L EQU 0CAH
RCAP2H EQU 0CBH
TL2 EQU 0CCH
TH2 EQU 0CDH
TR2 EQU T2CON.2
TF2 EQU T2CON.7
ET2 EQU IE.5

Handwritten notes: 50, 2, 00110100B

การส่งค่าแบบ Polling

B370' - -

```

MOV SBUF,A      ; Send Data
JNB TI,$        ; Wait Until Tx Complete
CLR TI          ; Clear Tx Flag

```

Handwritten notes: *mv A, #B'*, *sum 180*, *last*, *mv A, #13'*, *END*, *clr I*

การรับค่าแบบ Polling

```

CLR RI          ; Clear Rx Flag
JNB RI,$        ; Wait Until Rx Complete
MOV A,SBUF      ; Save Data

```

Handwritten notes: *RI = 12 RI*, *50*

การรับค่าแบบ Interrupts

```

SETB ES        ; Enable Serial Interrupts
SETB EA        ; Enable All

ORG 0023H      ; Serial Interrupt Vector 0023H
JMP SRI_ISR    ; Jump to ISR

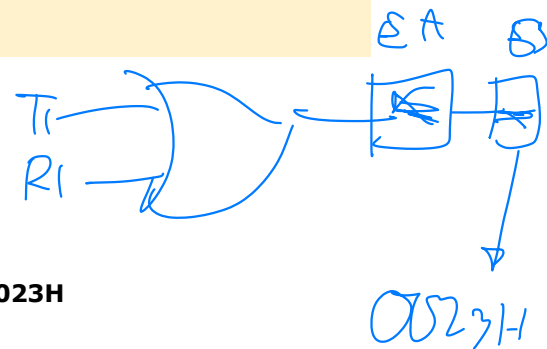
```

```

SRI_ISR:
CLR ES        ; Disable Serial Interrupts
JB TI,_X_SRI  ; If Tx Exit
MOV A,SBUF    ; Save Data
MOV SBUF,A    ; Echo Data
MOV Rx_Data,A ; Save to Variable

_X_SRI:
CLR TI        ; Clear Ti Flag
CLR RI        ; Clear Ri Flag
SETB ES       ; Enable Serial Interrupts
RETI          ; Return from Interrupts

```



3 คำถามในการทดลอง

1. ศึกษาการเขียนโปรแกรมเพื่อส่งค่าผ่าน Serial Port เพื่อแสดงผลที่หน้าจอโปรแกรม Hyper Terminal จากโปรแกรม “Lab10Srl1.Asm” และทดสอบการทำงานของโปรแกรม
 - ทดสอบการทำงานของโปรแกรมของโปรแกรม “Lab10Srl1.Asm”
 - 1.1 จากโปรแกรม “Lab10Srl1.Asm” ปรับให้บอร์ดทำหน้าที่ส่งข้อมูลไปยังเครื่องพีซี ด้วย Baud Rate = 9600 bps ให้โปรแกรมวนลูปส่งรหัสแอสกีออกไปยังเครื่องพีซีทีละ 1 ตัวทุก ๆ 0.5 วินาที(โดยประมาณ) ➔ “ 0,1,2,3,4,5,6,7,8,9,ขึ้นบรรทัดใหม่ และชิดซ้ายบรรทัด”
 - 1.2 จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ขึ้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11
2. ศึกษาการเขียนโปรแกรมเพื่อรับค่าจาก Serial Port ผ่านโปรแกรม Hyper Terminal จากโปรแกรม “Lab10Srl2.Asm” และทดสอบการทำงานของโปรแกรม โดยผลการทดสอบเมื่อกดค่า A หรือ B ที่ PC Keyboard ไมโครคอนโทรลเลอร์จะแสดงค่าที่ 7Segment Display
 - ทดสอบการทำงานของโปรแกรม “Lab10Srl2.Asm”
 - จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง - (ชิดกลาง)
 - จากโปรแกรม “Lab10Srl2.Asm” ถ้ารับค่าคีย์ 0, +, - หากเป็น + ให้ 7Segmentนับขึ้น, หากเป็น 0 ให้คงค่าการนับ และหากเป็น - ให้นับลง
3. การสร้างโปรแกรมน้อยๆเพื่อเรียกใช้
 - พิมพ์โค้ดโปรแกรมตามกรอบ “mySerial.A51” แล้วทำการบันทึกเป็น “mySerial.A51”
 - พิมพ์โค้ดโปรแกรมตามกรอบ “myDelay.A51” แล้วทำการบันทึกเป็น “myDelay.A51”
 - โปรแกรมหลักสามารถเรียกใช้ทุกชุดคำสั่งย่อยที่มีใน mySerial.A51 และ myDelay.A51ได้ทุกชุดแต่ต้องมีการประกาศบรรทัด \$Include (xxx.A51) ที่โปรแกรมหลักก่อน
 - ทดสอบการทำงานของโปรแกรม “Lab10Srl3.Asm”
 - จากโปรแกรม “Lab10Srl3.Asm” ปรับปรุงให้การนับ Counter เป็นแบบ BCD
4. การรับค่าแบบ Serial Interrupts
 - ทดสอบการทำงานของ “Lab10Srl5.Asm”
 - ปรับโปรแกรมให้รับค่าจาก Serial Terminal เพื่อคุม LED P3.7
 - ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ
 - ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ
5. การใช้งานด้วย Ride-C51
 - ทดสอบการทำงานของ “Lab10Srl4.C”
 - ปรับโปรแกรม Ride-C51 ให้รับค่าผ่าน Serial Terminal แล้วแสดงที่ 7Segment

4. คำถามท้ายการทดลอง

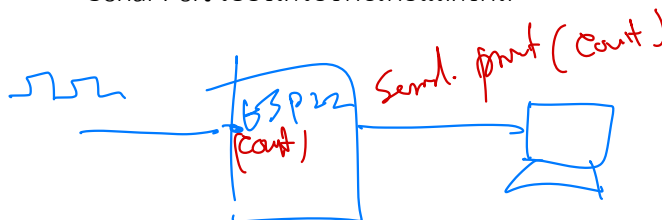
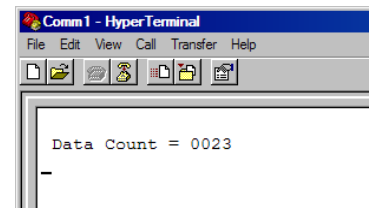
1. จงเขียนโปรแกรมเพื่อรับอินพุตจากสวิตช์ที่ต่อกับพอร์ต P2 จำนวนทั้งสิ้น 8 ตัว โดยกำหนดให้ P2.1 – P2.7 ทำหน้าที่กำหนดจำนวนบรรทัดของข้อมูลทั้งหมดที่จะส่งไปยังโปรแกรมไฮเปอร์เทอร์มินอล ดังนี้

พอร์ต	การทำงาน
P2.0 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบแบบไม่รู้จบ
P2.1 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 1 บรรทัด
P2.2 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 2 บรรทัด
P2.3 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 3 บรรทัด
P2.4 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 4 บรรทัด
P2.5 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 5 บรรทัด
P2.6 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 6 บรรทัด
P2.7 = 0	ให้ส่งตัวอักษร A ถึง Z แล้วขึ้นบรรทัดใหม่ วนรอบจำนวน 7 บรรทัด

2. ให้เขียนโปรแกรมสำหรับนับจำนวนชิ้นงานที่ไหลผ่านสายพานลำเลียงข้อสอบเก่า (แนวข้อสอบ)

Step 1: ทดสอบ Serial Port โดยให้ส่งข้อมูลไปยังเครื่องพีซี ด้วย Baud Rate = 9600 bps ให้โปรแกรมวนลูปส่งรหัสแอสกีออกไปยังเครื่องพีซีทีละ 1 ตัวทุก ๆ 0.5 วินาที ตามลำดับดังนี้ "0,1,2,3,4,5,6,7,8,9, ขึ้นบรรทัดใหม่ และชิดซ้ายบรรทัด"

Step 2: ทดสอบการนับโดยให้โปรแกรมควบคุม Timer 0 ให้ทำงานเป็น Counter mode 1 ใช้ตัวเลข HEX จากสัญญาณภายนอกนั้นนำค่า TH0TLO ส่งออกผ่าน Serial Port ไปยังเครื่องพีซีเพื่อแสดงผล



TH TL = 34 56
 TH TL

0011 0100
 3 A

Mov A, TH1 34
 SWAP A 43
 ANL A, #0FH 03
 ORL A, #30H 33

~~Mov A, #TH1~~

CLR TI
 Mov SBUF, A
 ZND TI, \$

Mov A, TH1 34
 ANL A, #0FH 04
 ORL A, #30H 34
 ~

Mov A, #TL1

CLR TI
 Mov SBUF, A
 ZND TI, \$

3A


```
ANL    A, #OF H
MOV    DPTR, # ASCII_TABLE
MOVC   A, A+DPTR
```

ASCII: DB '012345 . . . 89ABCDEF'

5 โปรแกรมทดสอบ

; Lab10Srl1.ASM >> Transmit Data

```

                ORG      0000H                      ; Lab10Srl1.Sub
                JMP      0100H

                ORG      0100H
                MOV      SP,#3FH
                CLR      EA                        ; Disable All
                SETB     P1.0                      ; Input Pin
                JB       P1.0,$                    ; Wait Press P1.0
                CALL     Delay                     ; Start Delay
                MOV      TMOD,#00100001B          ; TIM2 T0M1
                MOV      SCON,#01010000B          ; Serial Mode1
                MOV      TH1,#0FBH                 ; 9600 FBH@18.432MHz
                SETB     TR1                       ; Start Timer1

Main_Loop:      MOV      A,#'A'
                MOV      SBUF,A
                JNB       TI,$
                CLR      TI
                CALL     Delay
                JMP      Main_Loop

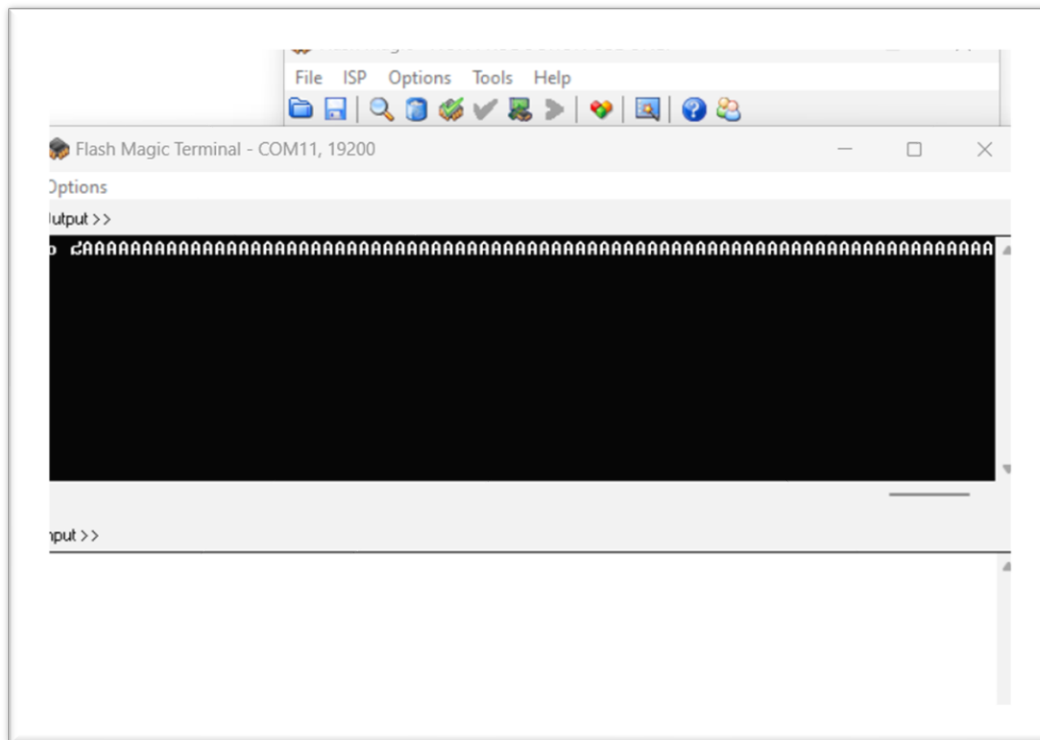
Delay:          CLR      A
                MOV      B,A
Dly00:          DJNZ     Acc,Dly00
                DJNZ     B,Dly00
                RET

                END

```

pinMode (circled around SETB P1.0)

Serial.begin(9600) (circled around TMOD, SCON, TH1, TR1)



; Lab10Srl2.ASM Polling Receive Data

```

T2CON      EQU      0C8H
RCAP2L     EQU      0CAH
RCAP2H     EQU      0CBH

                ORG      0000H                ; Lab10Srl2.Sub
                JMP      0100H

                ORG      0100H
                MOV      SP,#3FH
                CLR      EA                  ; Disable All
                MOV      SCON,#01010000B    ; Mode1 Serial Port
                MOV      RCAP2H,#0FFH       ; Reload values for desired
                MOV      RCAP2L,#0C4H       ; baud rate 9600@11.0592
                MOV      T2CON,#00110100B   ; Timer 2 as baud rate generator
                ; and turn on Timer 2

Main_Loop:    CLR      RI                  ; Ready for New Data
                JNB      RI,$               ; Wait Until Data In
                MOV      A,SBUF             ; Get Data In

_IS_ASCII_A:  CJNE      A,#'A',_IS_ASCII_B
                MOV      A,#0AH
                CALL     Display
                JMP      Main_Loop          Sh A

_IS_ASCII_B:  CJNE      A,#'B',_IS_ASCII_Etc
                MOV      A,#0BH
                CALL     Display
                JMP      Main_Loop          Sh B

_IS_ASCII_Etc: NOP
                JMP      Main_Loop

Display:      MOV      DPTR,#Seg_Tab
                MOVC     A,@A+DPTR
                CPL      A
                MOV      P0,A
                CLR      P1.4
                CLR      P1.5
                RET

Seg_Tab:      DB      07H, 11H, 6DH, 5DH, 1BH, 5EH, 7EH, 15H
                DB      7FH, 5FH, 3FH, 7AH, 66H, 79H, 6EH, 2EH
                ; 0 1 2 3 4 5 6 7
                ; i s a b c d e f.

                END

```



; 1/3 Edit and Save to "mySerial.A51"

```

;===== Initial Serial Port =====
Initial_Serial:  MOV     TMOD,#00100001B    ; T1M2 T0M1
                  MOV     SCON,#01010000B    ; Serial Mode1
                  MOV     TH1,#0FBH          ; 9600 FDH@18.432MHz
                  MOV     A,#00H             ; FBH@18.432MHz
                  MOV     PCON,A             ; SMOD = 0
                  CLR     EA                 ; Disable All
                  SETB    TR1                ; Start Timer1
                  RET

```

```

;===== Send Data to Serial Port =====

```

```

SEND_2HEX:  PUSH     Acc 1m
             SWAP     A
             CALL     SEND_1HEX

```

```

SEND_1HEX:  POP      Acc
             PUSH     DPH
             PUSH     DPL
             MOV      DPTR,#HEXASC_TAB
             ANL      A,#0FH
             MOVC     A,@A+DPTR
             POP      DPL
             POP      DPH

```

```

SEND_ASCII:  PUSH     IE
             CLR      ES
             CLR      TI
             MOV      SBUF,A
             JNB      TI,$
             CLR      TI
             POP      IE
             RET

```

```

HEXASC_TAB:  DB      '0123456789ABCDEF'

```

```

;===== Send Line Feed =====

```

```

Send_LineFeed:  MOV     A,#0DH
                 CALL    Send_ASCII
                 MOV     A,#0AH
                 CALL    Send_ASCII
                 RET

```

```

;===== Send Line Feed =====

```

```

Send_ClrScr:    MOV     A,#0CH
                 CALL    Send_ASCII
                 RET

```

```

;== Send Data Table to Serial Start@DPTR to '00H' =

```

```

Send_Table:     CLR     A
                 MOVC    A,@A+DPTR
                 JZ      _X_Send_Tab
                 CALL     SEND_ASCII
                 INC      DPTR
                 JMP      Send_Table
_X_Send_Tab:    RET

```

```

;===== Send 1-Byte@Acc to SERIAL =====

```

```

TX_BYTE:        PUSH     IE
                 CLR      ES
                 CLR      TI
                 MOV      SBUF,A
                 JNB      TI,$
                 CLR      TI
                 POP      IE
                 RET

```

```

;===== Receive Data From SERIAL to Acc =====

```

```

RX_BYTE:        PUSH     IE
                 CLR      ES
                 JNB      RI,$
                 CLR      RI
                 MOV      A,SBUF
                 POP      IE
                 RET

```

```

; Wait data

```

3F
↓
ASCII m F
↓

; 2/3 Edit and Save to "myDelay.A51"

```

#####
;# All Delay for XTAL 18.432MHz 12Cik/MC
;#
DELAY_4SEC:    PUSH    07H                ; Delay 80*50MilliSec
               MOV     R7,#80            ; = 4000MilliSec
               JMP     _DELAY_50M

DELAY_2SEC:    PUSH    07H                ; Delay 40*50MilliSec
               MOV     R7,#40            ; = 2000MilliSec
               JMP     _DELAY_50M

DELAY_1SEC:    PUSH    07H                ; Delay 20*50MilliSec
               MOV     R7,#20            ; = 1000MilliSec
               JMP     _DELAY_50M

DELAY_500M:    PUSH    07H                ; Delay 10*50MilliSec
               MOV     R7,#10            ; = 500MilliSec
               JMP     _DELAY_50M

DELAY_250M:    PUSH    07H                ; Delay 5*50 MilliSec
               MOV     R7,#5             ; = 250 MilliSec
               JMP     _DELAY_50M

DELAY_100M:    PUSH    07H                ; Delay 2*50 MilliSec
               MOV     R7,#2             ; = 100 MilliSec
               JMP     _DELAY_50M

DELAY_50M:     PUSH    07H                ; Delay 1*50 MilliSec
               MOV     R7,#1             ; = 50 MilliSec
               JMP     _DELAY_50M

_DELAY_50M:    PUSH    Acc
               PUSH    B
_DELAY_050M:    MOV     B,#200                ; Delay 50 MilliSec
               MOV     A,#191                ; 1MC ,250 uSec
_DELAY_250U:    DJNZ    Acc,$                ; 2MC ,(192-1)*0.651*2 = 250 uSec
               DJNZ    B,_DELAY_250U        ; 250*200=50mS
               DJNZ    R7,_DELAY_050M      ; 50*5 =250mS
               POP     B
               POP     Acc
               POP     07H
               RET

```

3/3 Edit and Save to "Lab10Srl3.ASM"

```

T2CON      EQU      0C8H
RCAP2L     EQU      0CAH
RCAP2H     EQU      0CBH

Counter    EQU      40H          ; Lab10Srl3.Sub
PortO_LEDGreen EQU      P3.7

          ORG      0000H
          JMP      0100H

          ORG      0100H
          MOV      SP,#3FH
          CLR      EA
          CALL     Delay_250M
          CALL     Initial_Serial
          CALL     Send_ClrScr
          MOV      Counter,#00
Main_Loop: MOV      DPTR,#Header_Fram
          CALL     Send_Table
          MOV      A,Counter
          CALL     Send_2HEX
          CALL     Send_LineFeed
          CPL      PortO_LEDGreen
          INC      Counter
          CALL     Delay_500M
          JMP      Main_Loop

$INCLUDE (myDelay.A51)
$INCLUDE (mySerial.A51)

Header_Fram: DB      'Data Counter = ',00
             END

```

Flash Magic Terminal - COM11, 19200

Options

Output >>

```

Data Counter = 2B
Data Counter = 2C
Data Counter = 2D
Data Counter = 2E
Data Counter = 2F
Data Counter = 30
Data Counter = 31
Data Counter = 32
Data Counter = 33
Data Counter = 34

```

RideC51 >> "Lab10Srl4.C"

```

#include <reg52.h>
#include <stdio.h>

sbit LED_0 = P3 ^ 6;
sbit LED_1 = P3 ^ 7;
sbit CS_Seg0 = P1 ^ 4;
sbit CS_Seg1 = P1 ^ 5;
#define PortO_DispData P0
unsigned int Count, Counter = 0;
char myString[80];
code unsigned char CodeSeg[] =
{ 0x77, 0x11, 0x6D, 0x5D, 0x1B, 0x5E, 0x7E, 0x15,
  0x7F, 0x5F, 0x3F, 0x7A, 0x66, 0x79, 0x6E, 0x2E
};

void Delay(unsigned char msec)
{ unsigned char i, j;
  while (msec)
  { for (i = 1; i <= 250; i++) //not :
    for (j = 1; j <= 113; j++) //with ;
      msec--;
  }
}

void Timer0_ISR(void) interrupt 1
{ unsigned char Temp;
  if (Count % 2 == 0) {
    Temp = Counter & 0x0f;
    CS_Seg0 = 1;   CS_Seg1 = 0;
  }
  else {
    Temp = (Counter >> 4) & 0x0f;
    CS_Seg0 = 0;   CS_Seg1 = 1;
  }
  PortO_DispData = ~CodeSeg[Temp];
  Count++;
}

void uart_tx(unsigned char serialdata) {
  SBUF = serialdata; //Load Data to Serial Buffer Register
  while (TI == 0); //Wait Until Transmission To Complete
  TI = 0; //Clear Transmission Interrupt Flag
}

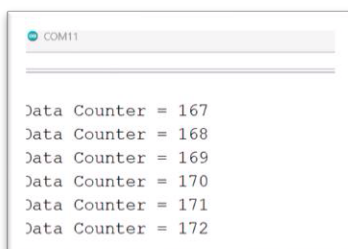
void uart_msg(unsigned char * c) {
  while ( * c != 0) {
    uart_tx( * c++);
  }
}

//=====
main()
{ TMOD = 0x22; // Time1Mode2 Timer0Mode1

  SCON = 0x50;
  TH1 = 0xFB; // Baud=9600 @18.432MHz x1
  TR1 = 1;

  TH0 = 0;
  TR0 = 1;
  ET0 = 1; // Enable External 0 Interrupt
  EA = 1; // Global Interrupt Enable
  while (1)
  { Delay(50);
    Counter++;
    sprintf(myString, "\nData Counter = %d", Counter);
    uart_msg(myString);
    LED_0 = (Counter + 1) % 2;
    LED_1 = Counter % 2;
  }
}

```



```

COM11

Data Counter = 167
Data Counter = 168
Data Counter = 169
Data Counter = 170
Data Counter = 171
Data Counter = 172

```

Interrupt	Name	Numbers
External Interrupt 0	(INT0)	0
Timer Interrupt 0	(TF0)	1
External Interrupt 1	(INT1)	2
Timer Interrupt 1	(TF1)	3
Serial Communication	(RI + TI)	4
Timer 2 (8052 only)	(TF2)	5

Edit and Save to "Lab10Srl5.ASM" Receive via Serial Interrupt

```

T2CON      EQU      0C8H
RCAP2L     EQU      0CAH
RCAP2H     EQU      0CBH
RX_Data    EQU      30H

          ORG      0000H
          JMP      0100H

          ORG      0023H
          JMP      Serial_ISR

          ORG      0100H
          MOV      SP,#3FH
          CLR      EA
          CALL     Delay_250M
          CALL     Initial_Serial
          CALL     Send_ClrScr
          SETB     ES
          SETB     EA

Main_Loop: CALL     Display
          JMP      Main_Loop

Display:   MOV      DPTR,#Seg_Tab
          MOV      A,RX_Data
          SETB     P1.4
          CLR      P1.5
          CALL     _Display
          MOV      A,RX_Data
          CLR      P1.4
          SETB     P1.5
          SWAP     A
          ANL      A,#0FH
          MOVC     A,@A+DPTR
          CPL      A
          MOV      P0,A
          CLR      A
          DJNZ     Acc,$
          RET

_Seg_Tab:  DB      77H, 11H, 6DH, 5DH, 1BH, 5EH, 7EH, 15H
          DB      7FH, 5FH, 3FH, 7AH, 66H, 79H, 6EH, 2EH

Serial_ISR: CLR      ES                ; Disable Serial Interrupts
          JB      TI,_X_Srl           ; If Tx Exit
          MOV     A,SBUF              ; Save Data
          MOV     SBUF,A              ; Echo Data
          MOV     Rx_Data,A           ; Save to Variable
          CLR     TI                  ; Clear Ti Flag
          CLR     RI                  ; Clear Ri Flag
          SETB     ES                 ; Enable Serial Interrupts
          RETI                        ; Return from Interrupts

_X_Srl:    CLR      TI
          CLR      RI
          SETB     ES
          RETI

$INCLUDE (myDelay.A51)
$INCLUDE (mySerial.A51)

END

```



4/7: ESP32 Serial Communication

Test21 - Get Data from Serial Port with Serial Read

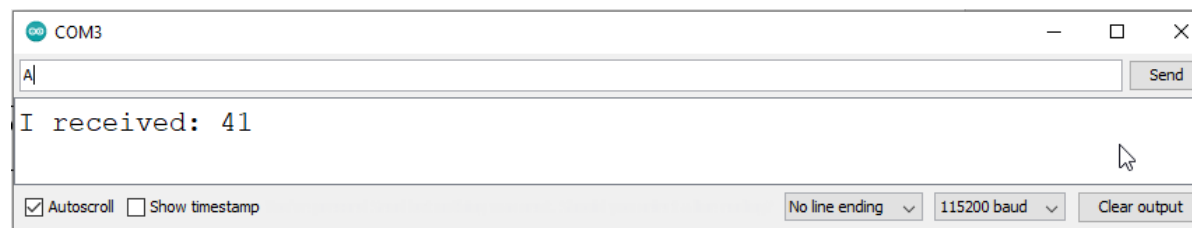
```
int incomingByte = 0; // for incoming serial data

void setup() {
  Serial.begin(115200); // opens serial port, sets data rate to 115200 bps
}

void loop() {
  // reply only when you receive data:
  if (Serial.available() > 0) {
    // read the incoming byte:
    incomingByte = Serial.read();

    // say what you got:
    Serial.print("I received: ");
    Serial.println(incomingByte, HEX);
  }
}
```

ASCII



Test22 - Get Data from Serial Port with Serial parseInt

```
int incomingData = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    incomingData = Serial.parseInt();
    Serial.print("I received: ");
    Serial.println(incomingData);
  }
}
```

int.



parseFloat

Test23 - Get 2Data from Serial Port with Serial.parseInt → separate number with space

```
int Data1, Data2;
char buffer[40];

void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    Data1 = Serial.parseInt();
    Data2 = Serial.parseInt();
    sprintf(buffer, "Summ of %d and %d is %d", Data1, Data2, Data1 + Data2);
    Serial.println(buffer);
  }
}
```

COM3

234 567

Summ of 234 and 567 is 801 ✓

☒ Autoscroll ☐ Show timestamp

No line ending 115200 baud Clear output

Test24 - Read String

```
String strData;
int Data1, Data2;

void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    strData = Serial.readString();
    Serial.println(strData);
  }
}
```

COM3

F=50

P=200

G

☒ Autoscroll ☐ Show timestamp

No line ending 115200 baud Clear output

Test25 - Read String and Split → P=xxx, F=yyy, G

```
String Cmd, strData;
int strInt, strlength, Pulse = 0, Freq = 0;
char buffer[40];

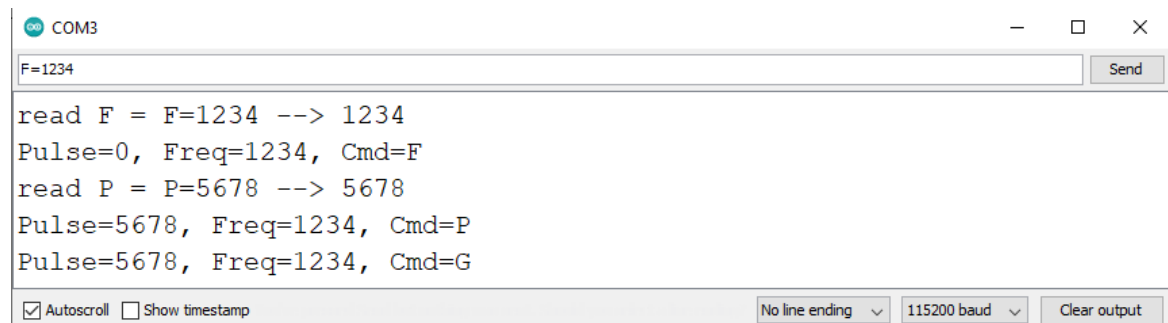
void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    strData = Serial.readString();
    strlength = strData.length();
    Cmd = strData.substring(0, 1);
    strInt = strData.substring(2, strlength).toInt();

    if (Cmd == "F") {
      if (strInt != 0) Freq = strInt;
      sprintf(buffer, "read F = %S --> %d", strData, strInt);
      Serial.println(buffer);
    }

    if (Cmd == "P") {
      if (strInt != 0) Pulse = strInt;
      sprintf(buffer, "read P = %S --> %d", strData, strInt);
      Serial.println(buffer);
    }

    sprintf(buffer, "Pulse=%d, Freq=%d, Cmd=%s", Pulse, Freq, Cmd);
    Serial.println(buffer);
  }
}
```



COM3

F=1234 Send

```
read F = F=1234 --> 1234
Pulse=0, Freq=1234, Cmd=F
read P = P=5678 --> 5678
Pulse=5678, Freq=1234, Cmd=P
Pulse=5678, Freq=1234, Cmd=G
```

☒ Autoscroll ☐ Show timestamp No line ending 115200 baud Clear output

Question-ESP32-01: เขียนโปรแกรมสร้างสัญญาณ โดยมีข้อกำหนดดังนี้

- รับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ P=xxx, D=yy, G, ? โดย
 - P=xxx<Cr>
 - ต้องมี Carriage return
 - กำหนดจำนวน Pulse ให้มีค่าระหว่าง 100-999
 - หากค่านี้เป็น 0 ให้ทำงานต่อเนื่องตลอดเวลา
 - D=yy<Cr>
 - ต้องมี Carriage return
 - กำหนดค่า Delay ให้มีค่าระหว่าง 10-99 หน่วยเป็น มิลลิวินาที
 - เมื่อป้อนแล้วให้ Echo แจ้งด้วยว่าจะได้ความถี่เท่าใด
 - G
 - ไม่ต้องมี Carriage return
 - ให้ ESP32 เริ่มทำงาน
 - ?
 - ไม่ต้องมี Carriage return
 - ให้ Echo แจ้งว่าใครเป็นคนเขียน โปรแกรมเวอร์ชันเท่าไร คอมมานอะไรบ้าง
- สร้างสัญญาณ Pulse ตามคำสั่งออกที่ D23
- เมื่อทำงานเสร็จให้ D23 = High
- ระหว่าง ESP32 ทำงานสามารถส่ง Serial Command ให้เปลี่ยนค่าได้เลย จำนวนพัลส์ได้

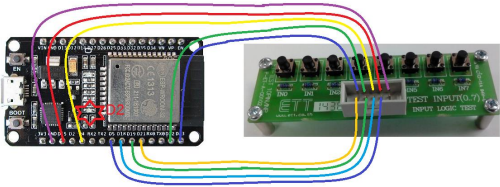
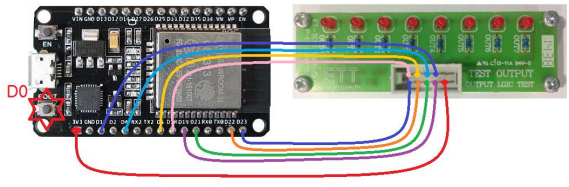
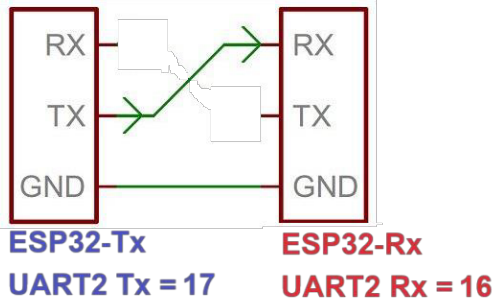
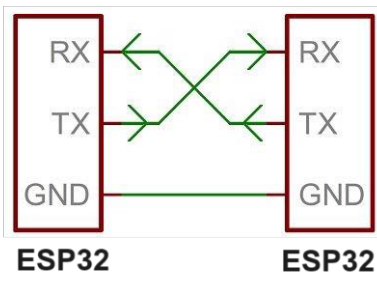
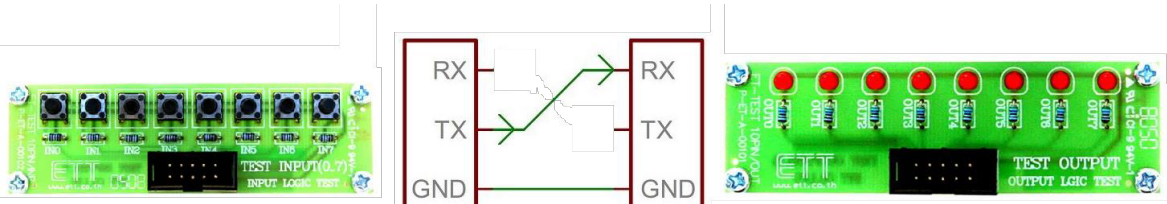
Question-ESP32-02: เขียนโปรแกรมส่ง-รับข้อมูล โดยมีข้อกำหนดดังนี้

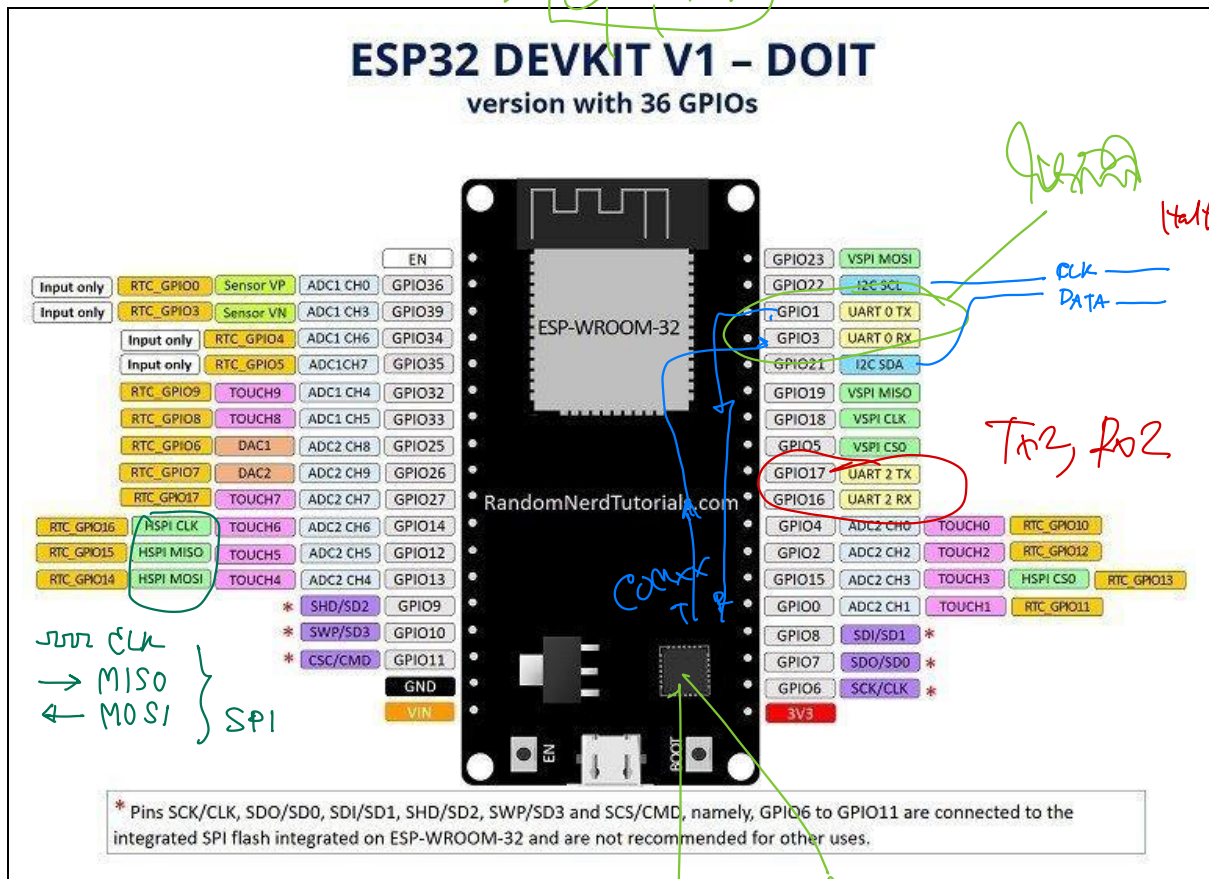
TX Board: ให้เป็นบอร์ดอ่านค่าสวิตช์แล้วส่งข้อมูลออก Serial

- ต่อยวงจรตาม “Digital Input – 8 Sw”, ใช้ Serial2 ในการส่งข้อมูล
- วงจรตรวจสอบการกดสวิตช์ที่ต่อกับ ESP32 Rx Board จำนวน 8 ตัว
- หากมีการกดสวิตช์ใดให้ ส่งรหัส ASCII เลขสวิตช์ไปยังบอร์ด RX พร้อมทั้งแสดงเลขสวิตช์ที่ Serial Terminal (เว้นขยาย) ของบอร์ดส่ง

RX Board: ให้เป็นบอร์ดรับค่าจาก Serial แล้วแสดงที่ LED

- ต่อยวงจรตาม “Digital Output – 8 LED”, ใช้ Serial2 ในการรับข้อมูล
- วงจรบรรับค่าจาก ESP-32 Tx Board
- นำค่ารหัส ASCII ที่รับได้แสดงผลที่ Serial Terminal (เว้นขยาย) ของบอร์ดรับ และ 8 LED Board

Tx-Board 	Rx-Board 
Tx-Rx System – One Way  ESP32-Tx UART2 Tx = 17 ESP32-Rx UART2 Rx = 16	Tx-Rx System – Two Way  ESP32 ESP32
 ESP32-Tx UART2 Tx = 17 ESP32-Rx UART2 Rx = 16	



Serial2.setpin(22Rx, 22Tx)

Ser 2. print/n

5/7: ESP32 OTA (Over the Air) Programming

- <https://lastminuteengineers.com/esp32-ota-updates-arduino-ide/>
- <https://lastminuteengineers.com/esp32-ota-web-updater-arduino-ide/>
- <https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/>
- <https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/>

Ways To Implement OTA In ESP32

There are two ways to implement OTA functionality in ESP32.

- Basic OTA – Over-the-air updates are sent through Arduino IDE.
- Web Updater OTA – Over-the-air updates are sent through a web browser.

Test31 – Basic OTA

- <https://lastminuteengineers.com/esp32-ota-updates-arduino-ide/>

Test32 – Web Updater OTA

- <https://lastminuteengineers.com/esp32-ota-web-updater-arduino-ide/>

Test33 – IAP (In Application Programming)

- <https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/>

Test34 – Blynk OTA

- <https://docs.blynk.io/en/blynk.edgent/updating-devices-firmwares-ota>

ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี

Blink01	Blink02
<pre>void setup() { pinMode(LED_BUILTIN, OUTPUT); } void loop() { digitalWrite(LED_BUILTIN, HIGH); delay(500); digitalWrite(LED_BUILTIN, LOW); delay(500); }</pre>	<pre>void setup() { pinMode(LED_BUILTIN, OUTPUT); } void loop() { digitalWrite(LED_BUILTIN, HIGH); delay(250); digitalWrite(LED_BUILTIN, LOW); delay(250); digitalWrite(LED_BUILTIN, HIGH); delay(250); digitalWrite(LED_BUILTIN, LOW); delay(1000); }</pre>

6/7: Industrial Ethernet Protocols

<https://www.encoder.com/wp2019-industrial-ethernet-communication-protocols>

1. Executive Summary

Industrial communications like all technologies, have undergone significant changes. Industry was once dominated by serial networks that utilized the technology that served the contemporary needs of the marketplace. Higher speeds and greater throughput are now needed to facilitate new, more sophisticated slave devices, end-node equipment, together with new applications. As a result, industrial communications equipment has been transitioning to faster, deterministic Ethernet-based technology and communications protocols, yet no specific communications protocol has come to dominate the industry. In today's market slave communications are supported by a multiplicity of communications protocols, each providing its own set of capabilities and advantages.

2. Industrial Slave Equipment

Here, we will look at some past serial-based protocols together with industrial Ethernet protocols and learn how they fit into the industrial automation world, illustrated in Figure 1, to deliver real-time, determinism, and the low latency required for industrial communications.

Industrial slave equipment presents a host of challenges for manufacturers. Industrial slave equipment includes: programmable logic controllers (PLCs), industrial drives, sensors connected to fieldbuses, human/machine interface systems, grid infrastructure communications, industrial gateways, and other types of systems. Most equipment has been built around hardware-based protocol execution. Equipment manufacturers have had to support different hardware configurations of their products, each tuned to a specific protocol. This situation is triggering a transition for industrial slave communications, toward programmable software-based protocol execution, which enables multi-protocol industrial slave communication systems. Greater programmability is ushering in greater flexibility and agility for manufacturers, enabling them to offer cost-effective configurable solutions to their users who are deploying industrial networks, with a decrease in Total Cost of Ownership (TCO).

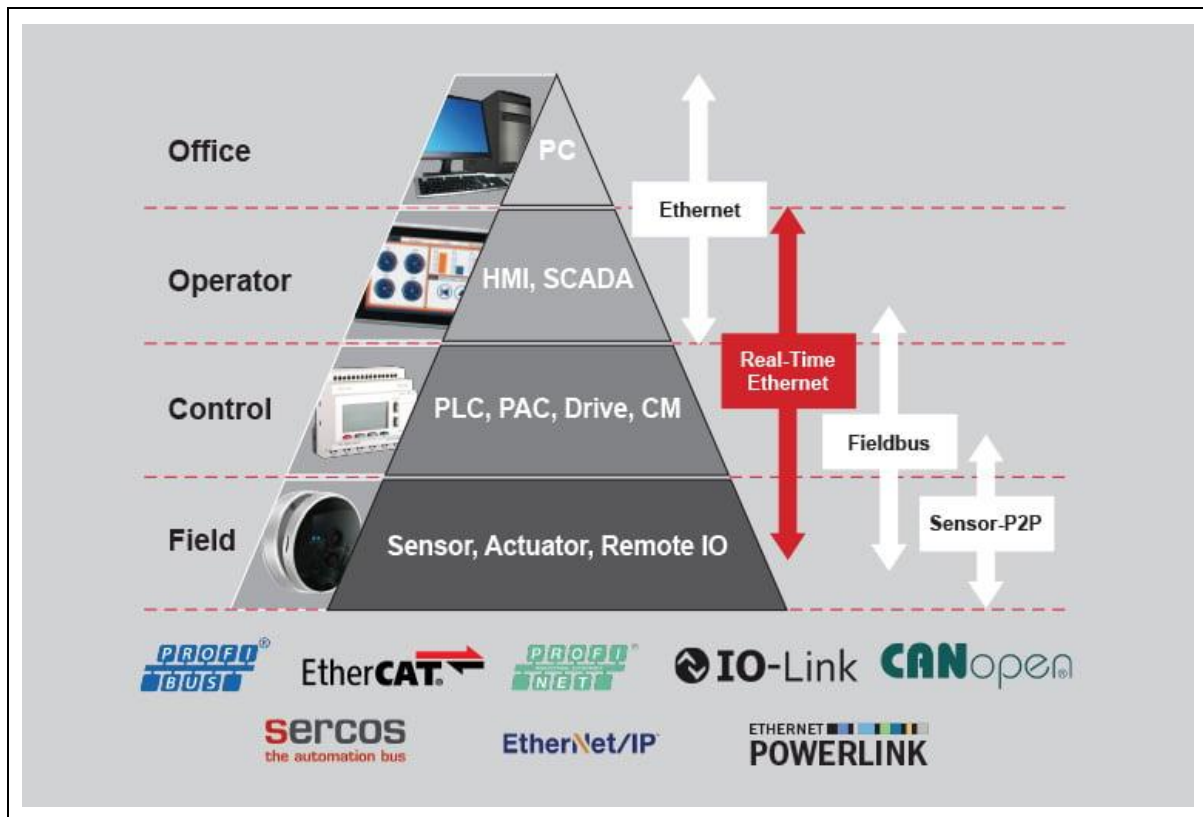


Figure 1. Industrial automation pyramid

3. Industrial Automation Components

Communication is the backbone of all industrial automation components for efficient production systems. There are four major components in industrial automation including PLC controllers, HMI panels, industrial drives, and sensors. Figure 2 illustrates these primary industrial automation components showing how all work together.

The PLC controller is the nexus of an industrial automation system; it provides relay control, motion control, industrial input and output process control, distributed system, and net-working control for the system.

The HMI is the graphical user interface for industrial control. It provides a command input and feedback output interface for controlling industrial machinery. An HMI is connected through common communication links to other parts of industrial systems.

Industrial drives are motor controllers used for controlling optimal motor operation. They are used in a very diverse range of industrial applications, and come with a wide range of voltage and power levels. Industrial drives include but are not limited to AC and DC drives as well as servo drives that use a motor feedback system to control and adjust the behavior and performance of servo mechanisms.

Sensors provide direct measurement for industrial automation systems that monitor the industrial operating conditions, inspections, position measurements, and much more, in real time. They are an integral part of industrial automation systems and provide trigger points and feedback for system control.

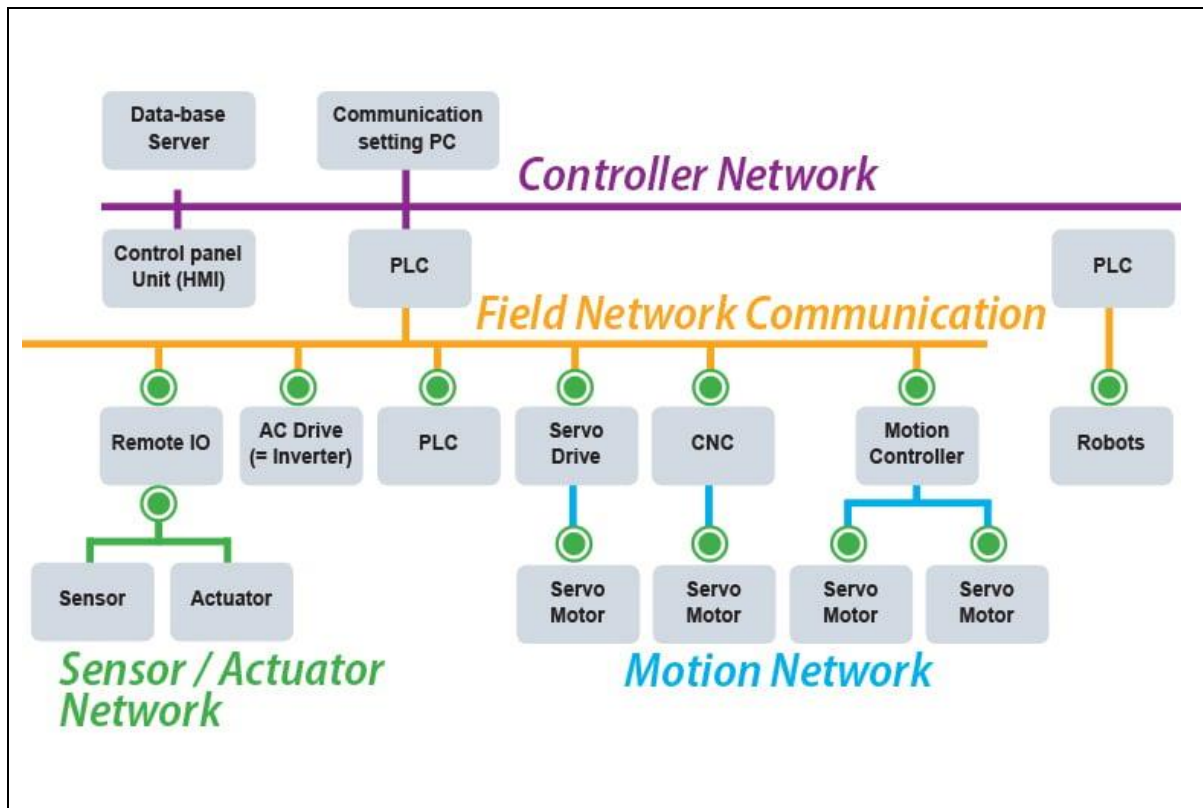


Figure 2. Industrial automation network

4. Legacy Industrial Communication Protocols

First generation industrial communications networks were built on a variety of serial based-interfaces. These interfaces were originally created by different companies that later became de facto standards. The companies that created these various serial-based interfaces were influential in promoting these de facto standards as well. This resulted in different standards in the marketplace. Large companies were behind these de-facto standards pushing them in the marketplace. Industrial automation equipment companies were then compelled to implement many of these protocols. Many serial-based protocols, including PROFIBUS, CAN bus, Modbus, CC-Link, and standard RS-422/RS-485 remain popular today due to the long life cycles of industrial systems.

These interfaces, many of which are still in use today, were slow and limited by the number of addresses (or number of slave nodes supported) on any one network segment. Throughput rates ranged upward to 12 megabits per second (Mbps) and a network could typically support no more than several hundred addresses for end nodes. These networks made effective use of the technology of the day and have stood the test of time. While the number of slave nodes supported is still considered sufficient, when combined with slow bus speeds, the time to exchange data between master and slave increases. This is especially critical as end nodes get more sophisticated with higher sensor integration, added intelligence, greater spatial separation, and eventually more data.

4.1 Profibus

PROFIBUS is widely deployed in industrial automation systems including those for factory and process automation. PROFIBUS provides digital communication for process data and auxiliary data with speeds up to 12 Mbps supporting up to 126 addresses.

4.2 CAN-bus

Control Area Network (CAN) bus, is a high-integrity serial bus system. It was originally created as an automotive vehicle bus and later came to be used as one of the fieldbuses for industrial automation. It provides a physical and data link layer for serial communication with speeds up to 1 Mbps. CANopen and DeviceNet are higher level protocols standardized on top of CAN bus to allow interoperability with devices on the same industrial network. CANopen supports 127 nodes on the network while DeviceNet supports 64 nodes on the same network.

4.3 Modbus

Modbus is a simple, robust and openly published, royalty free serial bus that connects up to 247 nodes together in the link. Modbus is easy to implement and runs on RS-232 or RS-485 with physical links speeds up to 115K baud.

4.4 CC-Link

CC-Link was originally developed by the Mitsubishi Electric Corporation in 1997 and is a popular open-architecture, industrial network protocol in Japan and Asia. CC-Link is based on RS-485 and can connect with up to 64 nodes on the same network with speeds up to 10 Mbps.

5. Descriptions of Industrial Ethernet Communication Protocols

Ethernet is cost effective and ubiquitous, offering common physical links with increased speed. As a result, many industrial communication systems are moving to Ethernet based solutions as illustrated in Figure 3. As the applications running on industrial networks became more sophisticated, they required greater and faster data transfer speeds. Ethernet based networking technology has provided a solution, as it is very cost effective, widely deployed, and understood. It is capable of much higher bandwidth performance than the serial fieldbus interfaces and scalable practically to an unlimited number of nodes. These capabilities translate into improved manufacturing yields, the ability to deploy new, more demanding applications and improved TCO.

However, before industrial Ethernet could be widely deployed, its primary constraint had to be overcome. Since the original Ethernet protocol relied on Collision Sense Multiple Access with Collision

Detection (CSMA/CD) mechanisms, it could not provide the determinism or predictability required by real-time communications applications such as industrial networking. Modifications could fortunately be made to Ethernet's Media Access Control (MAC) layer to facilitate real-time predictability and low latency response times. Now, several of the more popular real-time industrial Ethernet protocols will support networking speeds in the 100 megabits per second (Mbps) and the gigabit per second (Gbps) range going forward. The most prominent real-time Ethernet protocols today include EtherCAT, EtherNet/IP, PROFINET, POWERLINK, Sercos III, Modbus TCP, and CC-Link IE. Ethernet also enables flexible network topologies that scale with the number of network nodes.

Ethernet communications using standard TCP/UDP/IP are nondeterministic, with typical cycle times greater than 1000 ms, and packets subject to collision on the wire. Some Industrial Ethernet protocols use a modified Media Access Control (MAC) layer to achieve low latency and deterministic responses.

Table 1 illustrates the umbrella organizations responsible for each Industrial Ethernet type discussed here.

Figure 5 below illustrates how the Industrial Ethernet protocols relate to standard TCP/UDP/IP based Ethernet, together with their MAC layer modifications. Ethernet/IP, Modbus, and standard PROFINET protocols are completely TCP/UDP/IP based. Standard Ethernet switches and controllers can be used in the network (Figure 5a). For PROFINET RT and POWERLINK, process data is carried over TCP/UDP/IP with timing controlled by a process data driver. Standard Ethernet switches, hubs, and controllers can be used in the network (Figure 5b). For PROFINET IRT, CC-Link IE, SERCOS, and EtherCAT, process data is carried over TCP/UDP/IP with timing controlled by a process data driver. Special Ethernet link layer/MAC hardware is required for protocol slave devices to realize determinism (Figure 5c).

Nearly all Industrial Ethernet protocols, if not entirely TCP/UDP/IP based, provide some mechanism to encapsulate and carry standard Ethernet frames, (for example Ethernet-Over EtherCAT - EOE).

When using an Ethernet packet sniffer such as Wireshark, most Industrial Ethernet frames are identified by checking the ETHERTYPE ID eld. Table 2 illustrates how each of the different Industrial Ethernet types are detected using the Ethernet frame ETHERTYPE ID.

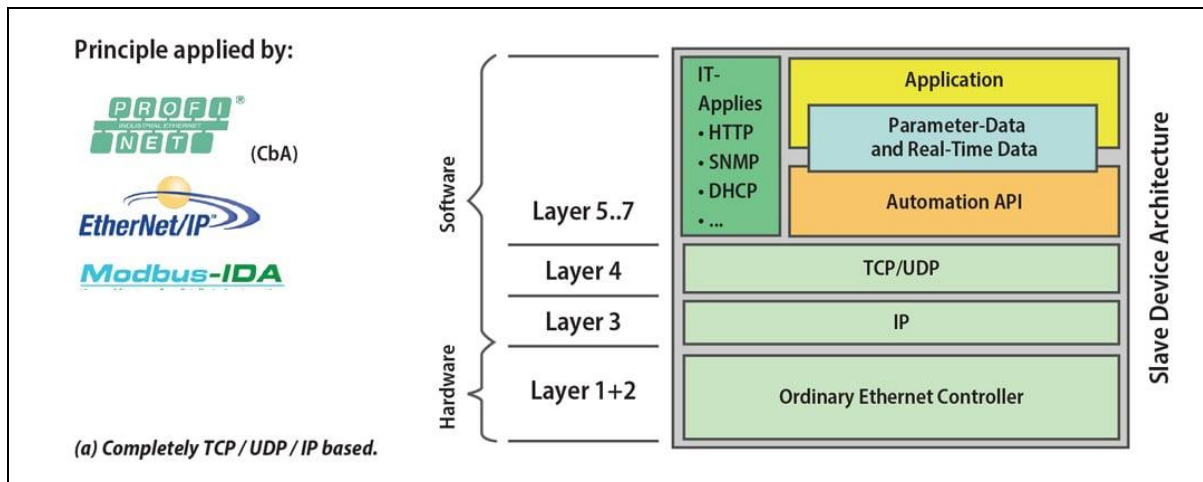


Figure 5a.

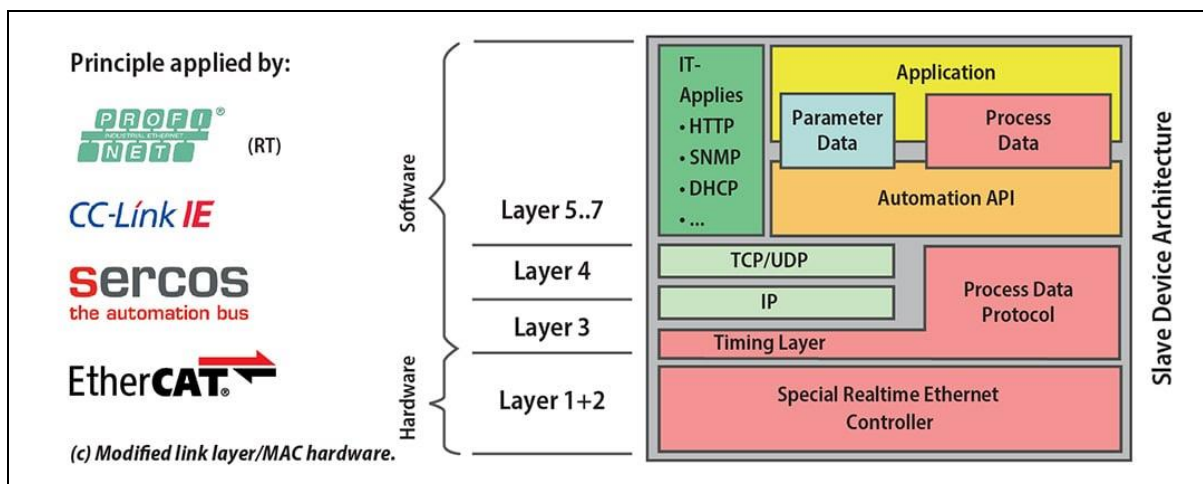


Figure 5b.

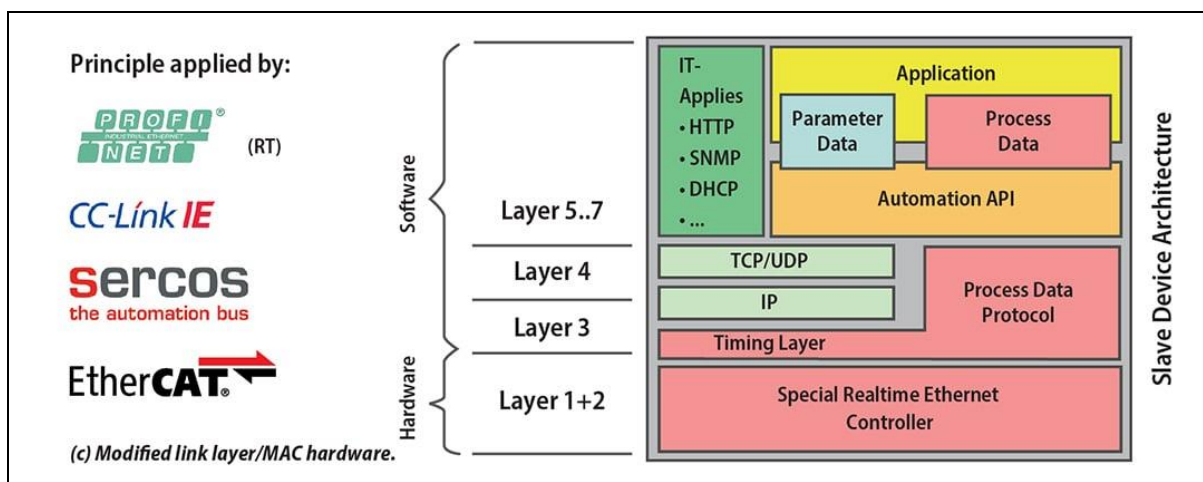


Figure 5c.

Figure 5. Basis for industrial ethernet protocols. Source: ETG

Table 1. Industrial ethernet organizations

Industrial Ethernet Protocol	Organization	Contact
PROFINET	PNO	probus.com
POWERLINK	ESPG	ethernet-powerlink.org
Ethernet/IP	ODVA	odva.org
EtherCAT	ETG	ethercat.org
SERCOS III	SERCOS International	sercos.org
MODBUS TCP	Modbus Organization	modbus.org
CC-Link IE	CLPA	cc-link.org

6. Ethernet/IP

EtherNet/IP is an industrial Ethernet protocol originally developed by Rockwell Automation that is now managed by the Open DeviceNet Vendors Association (ODVA). Unlike EtherCAT, which is a MAC-layer protocol, EtherNet/IP is an application-layer protocol built on top of TCP/IP. EtherNet/IP uses standard Ethernet physical, data link, network and transport layers, while using the Common Industrial Protocol (CIP) on top of TCP/IP. CIP provides a common set of messages and services for industrial automation control systems, where it can be used with multiple physical layer media. For example, CIP over CAN bus is called DeviceNet, CIP over a dedicated network is called ControlNet, and CIP over Ethernet is called EtherNet/IP. EtherNet/IP establishes communication from one application node to another through CIP connections over a TCP connection; multiple CIP connections can be established over one TCP connection.

EtherNet/IP uses standard Ethernet with switches, thus it can have an unlimited number of nodes in a system. This enables one network across many different end points in a factory floor. Ethernet/IP offers complete producer-consumer service and enables very efficient slave peer-to-peer communications. EtherNet/IP is compatible with many standard Internet and Ethernet protocols, but has limited real-time and deterministic capabilities without careful network planning.

7. PROFINET

PROFINET is widely used by major industrial equipment manufacturers such as Siemens and GE. It has three different classes: (1) PROFINET Class A provides access to a PROFIBUS network through proxy, bridging Ethernet and PROFIBUS with a remote procedure calling on TCP/IP. Its cycle time is approximately 100 ms, and is primarily used for parameter data and cyclic I/O. The typical application includes infrastructure and building automation. (2) PROFINET Class B, also referred as PROFINET Real-Time (PROFINET RT), introduces a software-based real-time approach and has reduced the cycle time to approximately 10 ms. Class B is typically used in factory automation and process automation. (3) PROFINET Class C (PROFINET IRT), is Isochronous and real-time, requiring special hardware to reduce the cycle time to less than 1ms to deliver sufficient performance on the real-time industrial Ethernet for motion control operations.

PROFINET RT can be used in PLC-type applications, while PROFINET IRT is a good fit for motion applications. Branch and Star are the common topology used for PROFINET. Careful topology planning is required for PROFINET networks to achieve the required performance of the system.

8. EtherCAT

EtherCAT was originally developed by Beckhoff to enable on-the-fly MAC packet processing and deliver real-time Ethernet to automation applications. It can provide scalable connectivity for entire automation systems, from large PLCs all the way down to the I/O and sensor level.

EtherCAT is a protocol optimized for process data, using standard IEEE 802.3 Ethernet Frames. Each slave node processes its datagram and inserts the new data into the frame while each frame is passing through. The process is handled in hardware (at the MAC layer) so each node introduces minimum processing latency, enabling the fastest possible response time. EtherCAT is a MAC layer protocol and is transparent to any higher level Ethernet protocols such as TCP/IP, UDP, Web server, etc. It can connect up to 65,535 nodes in a system. The EtherCAT master can be a standard Ethernet controller, thus simplifying the network configuration. Due to the low latency of each slave node, EtherCAT delivers flexible, low-cost and network-compatible industrial Ethernet solutions.

9. SERCOS III

Sercos III is the third generation of Serial Real-time Communication System (Sercos). It combines on-the-fly packet processing for delivering real-time Ethernet with standard TCP/IP communication to deliver low latency industrial Ethernet. Much like EtherCAT, a Sercos III slave processes the packet by extracting and inserting data to the Ethernet frame on-the-fly to achieve low latency. Sercos III separates input and output data into two frames. With cycle times from 31.25 microseconds, the speed rivals that of EtherCAT and PROFINET IRT. Sercos supports ring or line topology. Sercos III is limited to 511 slave nodes in one network and is most used in servo drive controls.

10. CC-Link IE

CC-Link IE is the industrial Ethernet technology of CC-Link, which was originally developed by the Mitsubishi Electric Corporation. CC-Link IE has two versions: CC-Link IE Control and CC-Link IE Field. CC-Link IE Control is intended for controller-to-controller communications and is limited to 120 nodes per network. CC-Link IE Field is intended for I/O communications and motion control, and is limited to 254 nodes per network. CC-Link IE leverages the Ethernet data link layer, and its control frames are directly embedded in the Ethernet frame. Only ring topology is supported in CC-Link, without switches. This can provide network redundancy, but only a limited number of nodes can be supported in a network, and the cycle time is dependent on the number of the nodes in the network.

11. Powerlink

POWERLINK was originally developed by B&R. Ethernet POWERLINK is implemented on top of IEEE 802.3 and, therefore, allows a free selection of network topology, cross connect and hot plug. It uses a polling and time slicing mechanism for real-time data exchange. A POWERLINK master or Managed Node controls the time synchronization through packet jitter in the range of tens of nanoseconds. Such a system is suitable for all kinds of automation systems ranging from PLC-to-PLC communication and visualization down to motion and I/O control. Barriers to implement POWERLINK are quite low due to the availability of open-source stack software. In addition, CANopen is part of the standard which allows for easy system upgrades from previous fieldbus protocols.

12. Modbus /TCP

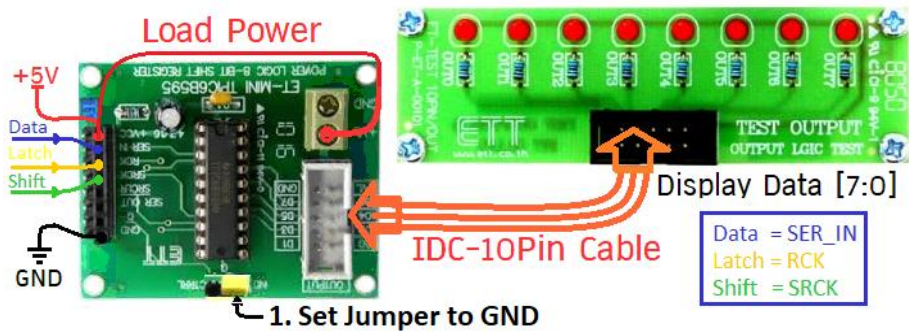
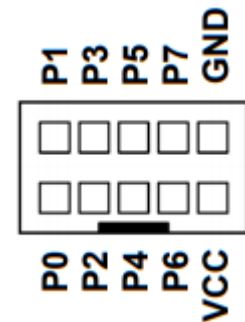
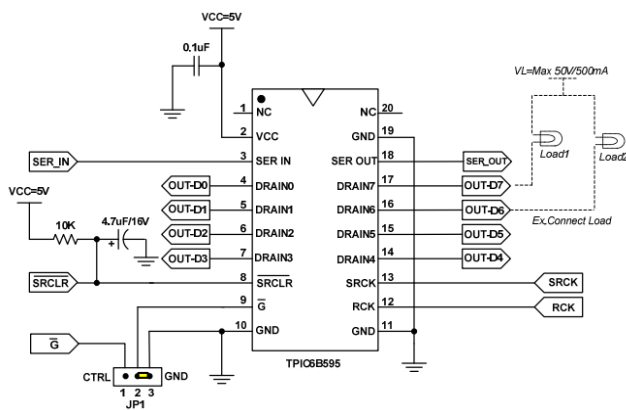
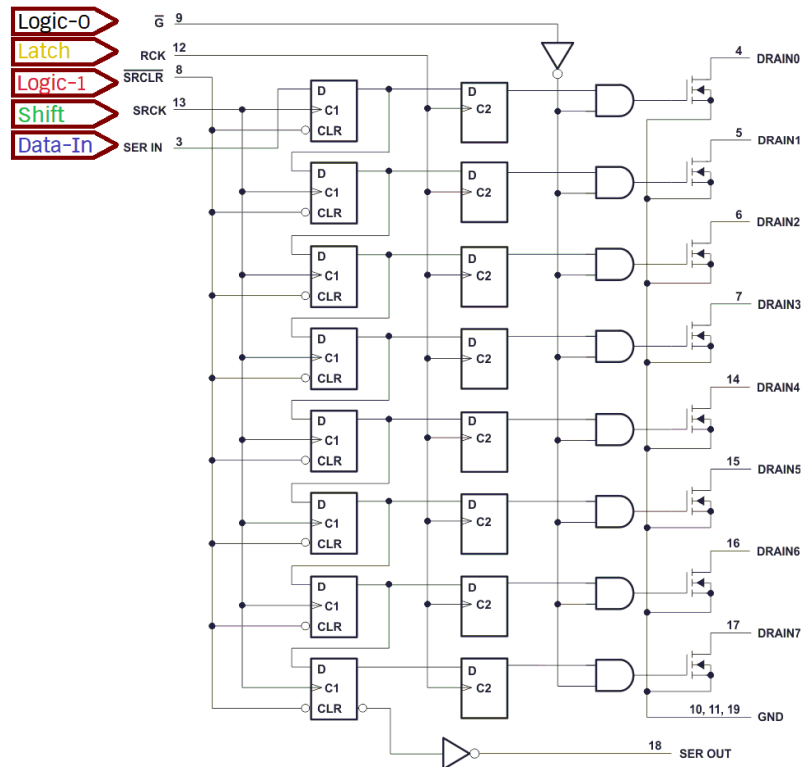
Modbus /TCP is an extension of serial Modbus®, and was originally developed by Schneider Electric. It uses serial Modbus messaging over TCP/IP on top of Ethernet. Modbus/TCP is simple to implement on the standard Ethernet network, but it does not guarantee real-time and deterministic communications: it is a non real-time approach to networking within the industrial environment. It does not guarantee arrive times or cycle times nor provide precise synchronization. Performance is a strong function of the underlying TCP/UDP/IP stack implementation.

13. Future Trends

As the third industrial revolution dawns, industrial automation will again drive the world economy. The success of industrial automation depends on a reliable and efficient communication network that connects all components of the factory seamlessly working together. The popularity and ubiquity of Ethernet will continue to motivate the legacy factory to upgrade to Industrial Ethernet. Many different Industrial Ethernet protocols have been implemented in the field, each with its strengths and weaknesses. Future Industrial Ethernet protocols will continue to evolve and converge to deliver hard real-time, deterministic communication links with better reliability and integrated safety. For more information on the EPC absolute Ethernet product line, please visit EPCs Encoders for Industrial Ethernet overview page.

7/7: Hardware

1. Shift Output 8Bit TPIC6B595 – Power Logic 8-bit Shift register



```

P_DATA EQU P2.7
P_LATCH EQU P2.6
P_SHIFT EQU P2.5

ORG 0000H
JMP 0100H

ORG 0100H
CLR EA
MOV SP,#3FH
SETB P_DATA
SETB P_LATCH
SETB P_SHIFT

LOOP:  MOV A,#10000000B
      CALL SEND_TPIC6B595
      MOV A,#01000000B
      CALL SEND_TPIC6B595
      MOV A,#00100000B
      CALL SEND_TPIC6B595
      MOV A,#00010000B
      CALL SEND_TPIC6B595
      MOV A,#00001000B
      CALL SEND_TPIC6B595
      MOV A,#00000100B
      CALL SEND_TPIC6B595
      MOV A,#00000010B
      CALL SEND_TPIC6B595
      MOV A,#00000001B
      CALL SEND_TPIC6B595
      MOV A,#11111111B
      CALL SEND_TPIC6B595
      MOV A,#00000000B
      CALL SEND_TPIC6B595
      MOV A,#11111111B
      CALL SEND_TPIC6B595
      CALL DELAY
      CALL DELAY
      JMP LOOP

SEND_TPIC6B595:
      MOV B,#8
      RLC A
      MOV P_DATA,C
      CLR P_SHIFT
      SETB P_SHIFT
      DJNZ B,_TXBit
      CLR P_LATCH
      SETB P_LATCH
      CALL DELAY
      RET

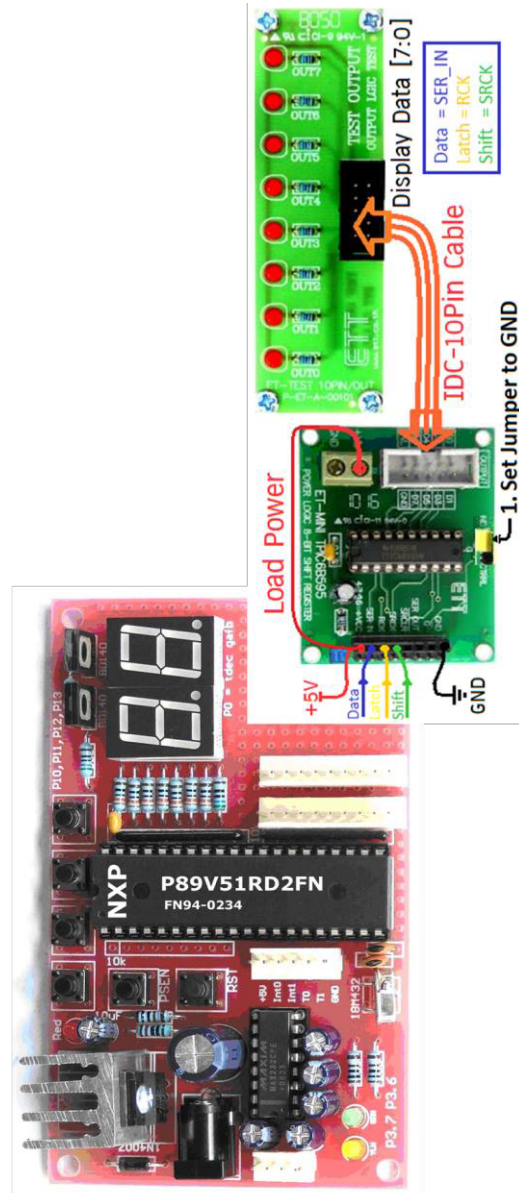
_TXBit:

DELAY:  CLR A
      MOV B,A
      DJNZ Acc,_DLY00
      DJNZ B,_DLY00
      RET

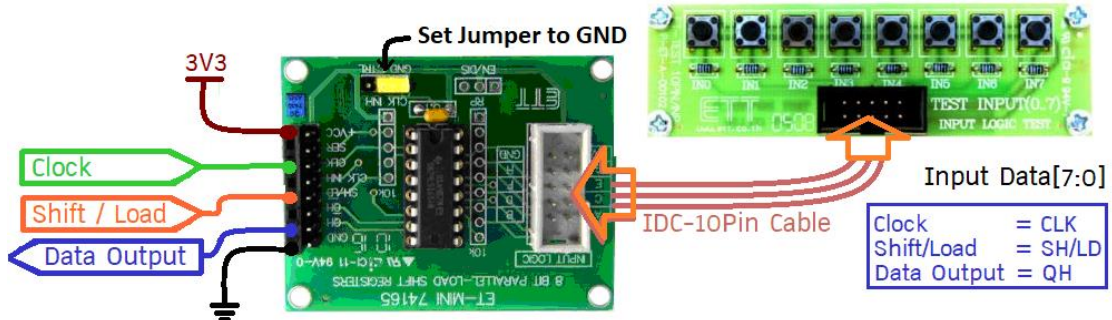
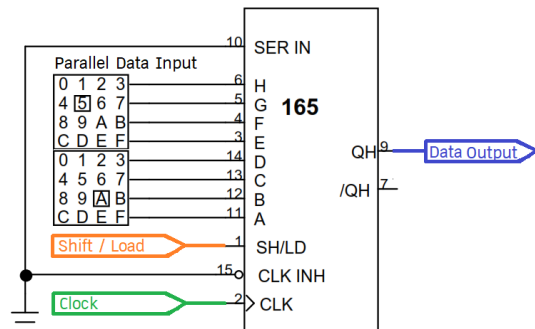
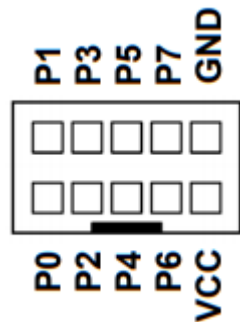
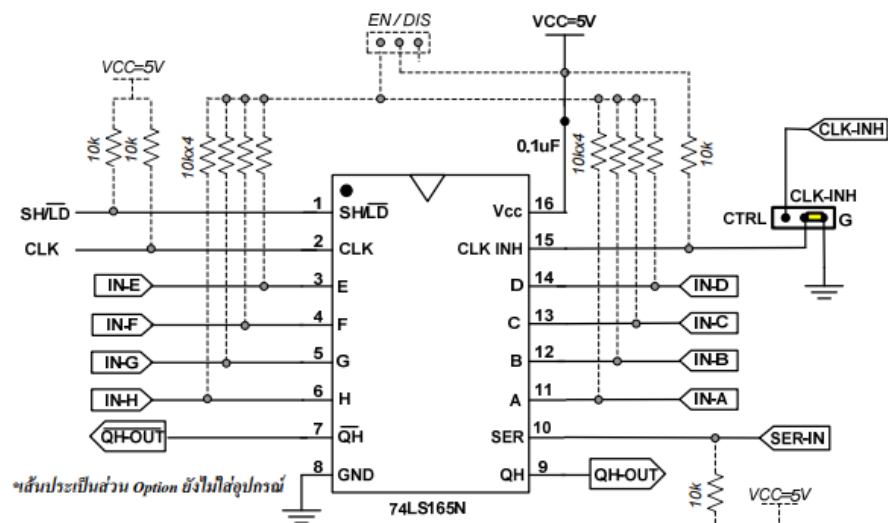
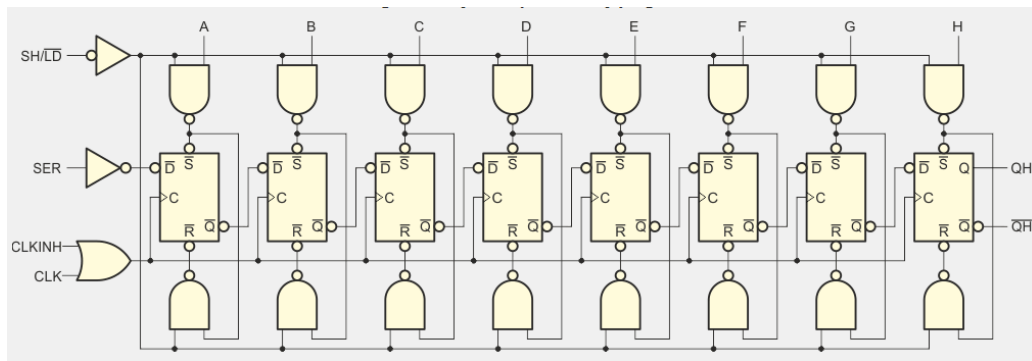
_DLY00:

END

```



2. Shift Input 8Bit: 74165 – Parallel Load Shift Register



```

P_DIN      EQU      P2.7
P_LOAD     EQU      P2.6
P_CLK      EQU      P2.5

ORG         0000H
JMP         0100H

ORG         0100H
CLR         EA
MOV         SP,#3FH
SETB       P_DIN
SETB       P_CLK
SETB       P_LOAD
CLR         P1.4      ; Show Left
CLR         P1.5      ; Show Right

LOOP:       CALL     GET_74165

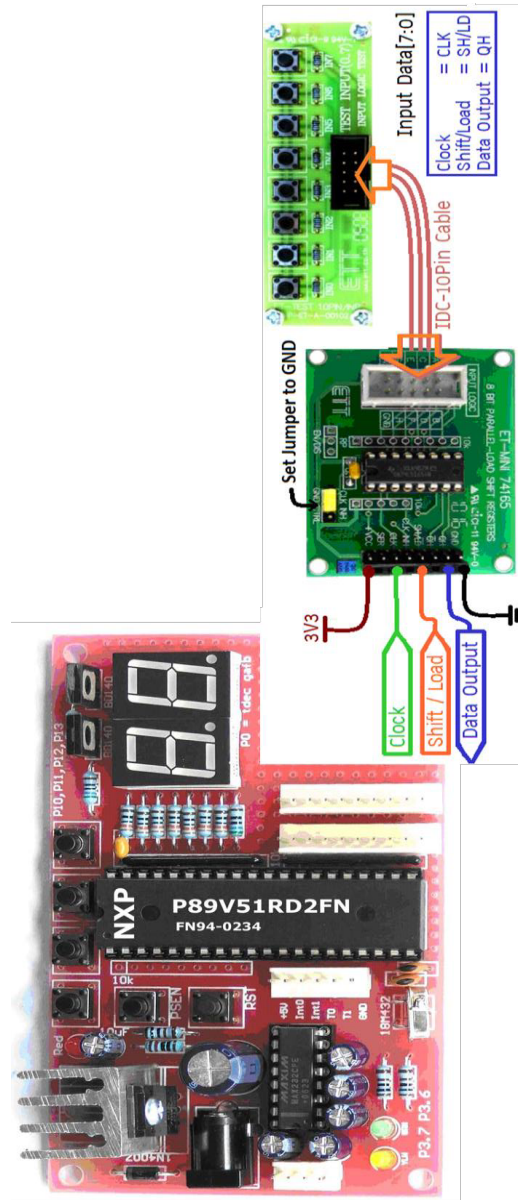
; A = TGFE DCBA >> TDEC GAFB = 7342 6051
RLC        A
MOV        P0.7,C      ; T
RLC        A
MOV        P0.3,C      ; G
RLC        A
MOV        P0.1,C      ; F
RLC        A
MOV        P0.5,C      ; E
RLC        A
MOV        P0.6,C      ; D
RLC        A
MOV        P0.4,C      ; C
RLC        A
MOV        P0.0,C      ; B
RLC        A
MOV        P0.2,C      ; A

JMP        LOOP

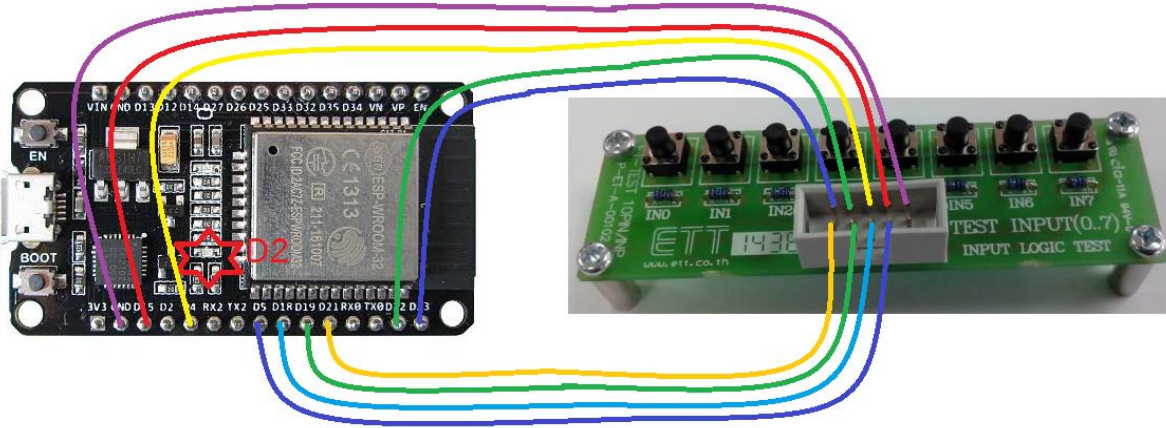
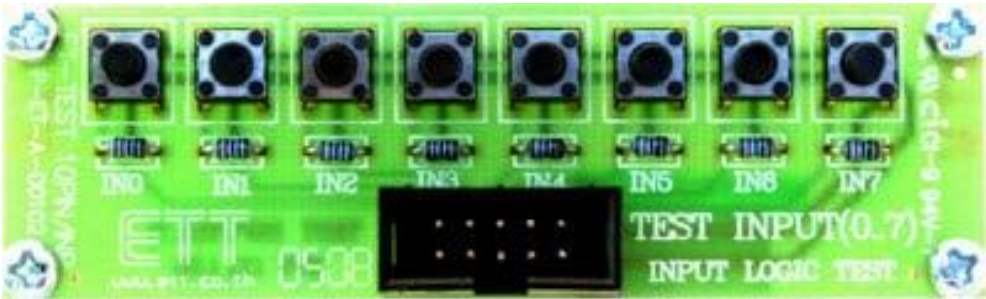
GET_74165:
MOV        B,#8
CLR        P_LOAD
SETB       P_LOAD
MOV        C,P_DIN
RLC        A
CLR        P_CLK
SETB       P_CLK
DJNZ       B,_RXBit
RET

_RXBit:
END

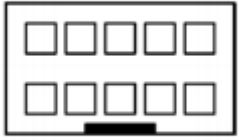
```



3. Digital Input – 8 SW

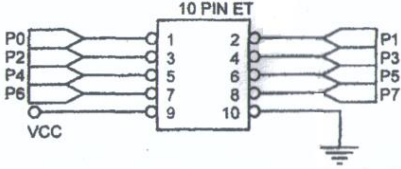



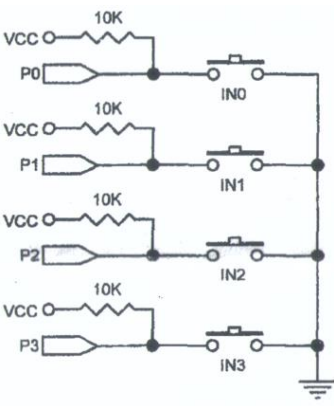
P1 P3 P5 P7 GND

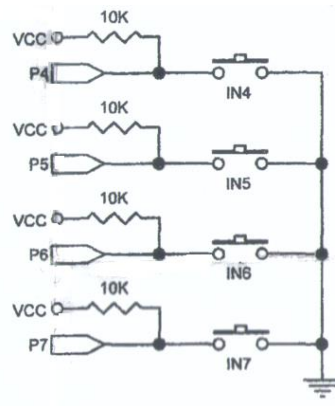


P0 P2 P4 P6 VCC

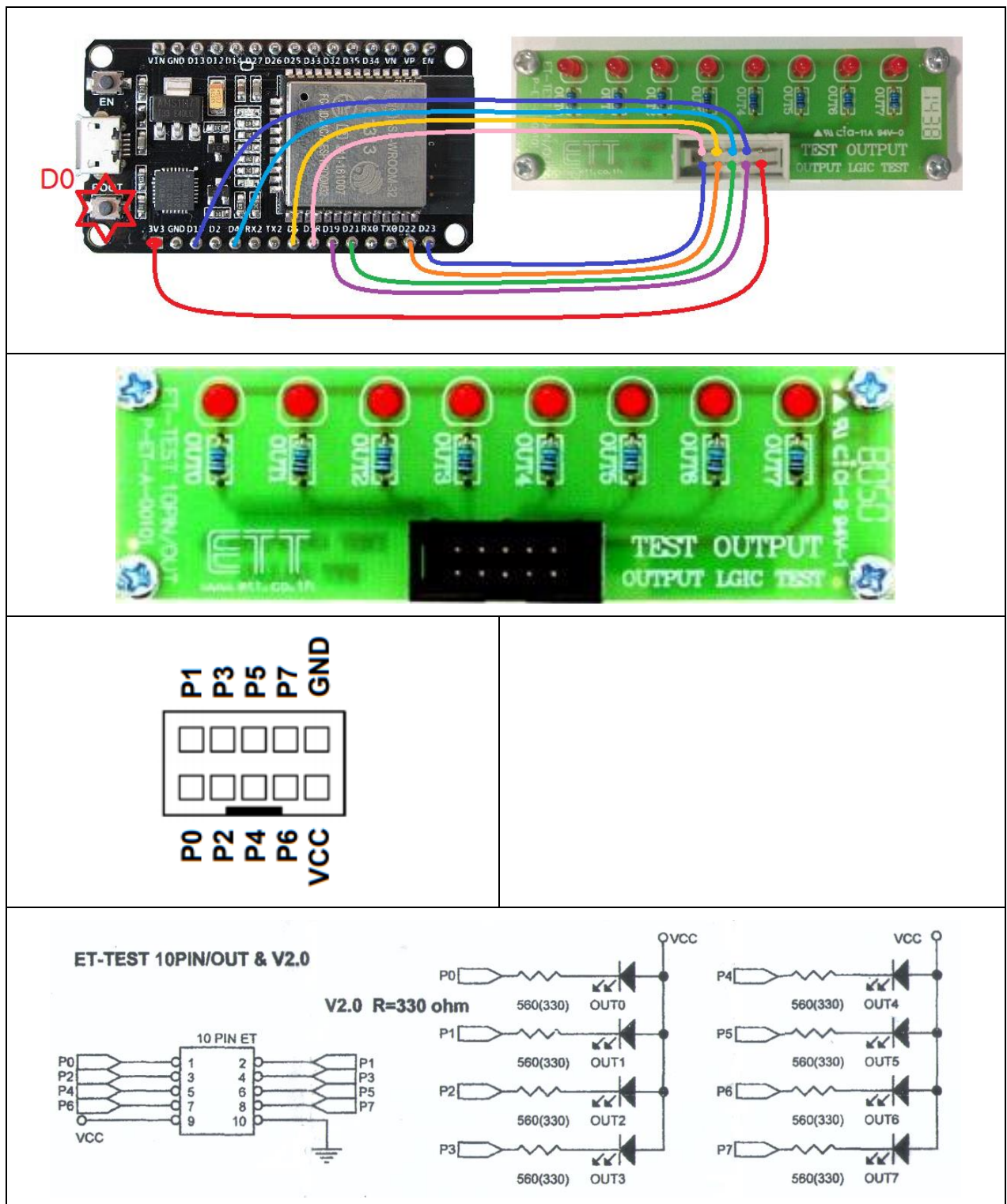
ET-TEST 10PIN/INP







4. Digital Output – 8 LED



5. Digital Output – 7_Segment

560 Ohm = Grn Ble Blk Blk Brw

	abcdef		bcfg		abcdefg		deg
	bc		acd fg		abcdfg		bcdeg
	abdeg		acdefg		abcefg		adefg
	abcdg		abcf		cdefg		aefg